

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 24 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory → blocks/SM; block 大小 → blocks/SM
49 //
50 // ---- 常见优化手段速查清单 ----
51 //
52 // 1. Coalesced Memory Access (合并访问)
53 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
54 //   - 否则产生多次事务 (最坏 32 次)
55 //
56 // 2. Tiling (分块)
57 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
58 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
59 //
60 // 3. Vectorized Memory Access (向量化)
61 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数
62 //   - float4 = 128-bit, 单条指令加载 16 bytes

```

```

63 //
64 // 4. Thread Tile (寄存器分块)
65 //   - 每个线程计算多个输出元素 (TM×TN)，提高计算密度
66 //   - 减少线程总数，降低同步开销
67 //
68 // 5. Bank Conflict Avoidance
69 //   - Shared memory 有 32 banks × 4 bytes
70 //   - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
71 //   - 解决方案：PAD (在每行末尾加 1 个元素打破对齐)
72 //
73 // 6. Pipeline / Double Buffering (流水线)
74 //   - cp.async 异步拷贝下一批数据时同时做当前批的计算
75 //   - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
76 //
77 // 7. Tensor Core (MMA / WGMMMA)
78 //   - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
79 //   - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
80 //
81 // 8. Warp Specialization (Hopper+)
82 //   - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
83 //   - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
84 //
85 // 9. TMA (Tensor Memory Accelerator, Hopper+)
86 //   - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
87 //   - 配合 cp.async.bulk 实现异步数据搬运
88 //
89 // ---- Roofline 分析公式 ----
90 //
91 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
92 //
93 // GEMM (M=N=K=4096):
94 //   FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
95 //   Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
96 //   AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
97 //   FP16 TC ≈ 295:1, FP32 ≈ 20:1)
98 //
99 // GEMV (M=4096, K=4096):
100 //   FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
101 //   Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 //   AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105 //
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 //   - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 //   - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //     注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 //   - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115 //
116 #include <algorithm>
117 #include <cuda_fp16.h>
118 #include <cuda_runtime.h>
119 #include <float.h>
120 #include <stdio.h>
121 #include <stdlib.h>
122 #include <math.h>
123 #include <cublas_v2.h>
124 #include <cuda.h>
125 #include <cuda/barrier>

```

```

126
127 #define WARP_SIZE 32
128 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
129 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
130 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
131
132 // =====
133 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
134 // =====
135
136 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
137 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
138 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
139 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
140 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
141 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
142 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
143 //
144 // 初始: 每个 lane 持有自己的值 v0..v7
145 // lane:  0    1    2    3    4    5    6    7
146 // val:  v0   v1   v2   v3   v4   v5   v6   v7
147 //
148 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
149 //
150 //      ┌──────────┐
151 // 对:   (0,4) (1,5) (2,6) (3,7)
152 //
153 // lane:  0    1    2    3    4    5    6    7
154 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
155 //
156 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
157 //
158 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
159 // 对:   (0,2) (1,3) (4,6) (5,7)
160 //      ─── 前4个一组 ───    ─── 后4个一组 ───
161 //
162 // lane:  0    1    2    3    4    5    6    7
163 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$  (每lane持有前一轮2个值的和再加本轮配对)
164 //      = v0+v2+v4+v6 ... (逐步归约)
165 //
166 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
167 //
168 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
169 // 对:   (0,1) (2,3) (4,5) (6,7)
170 //
171 // lane:  0    1    2    3    4    5    6    7
172 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
173 //
174 // mask=0: 循环终止, 归约完成。
175 //
176 // 关键性质:
177 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
178 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
179 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
180 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
181 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
182 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
183 template <const int kWarpSize = WARP_SIZE, typename T = float>
184 __device__ __forceinline__ T warp_reduce_sum(T val) {
185 #pragma unroll
186   for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
187     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
188   }
189   return val;
190 }

```

```

188
189 // Warp Reduce Max — generic
190 template <const int kWarpSize = WARP_SIZE, typename T = float>
191 __device__ __forceinline__ T warp_reduce_max(T val) {
192 #pragma unroll
193     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
194         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
195     }
196     return val;
197 }
198
199 // =====
200 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
201 // =====
202
203 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
204 // 两级归约流程:
205 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
206 // 2. warp leader (lane=0) 写入 shared memory
207 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
208 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
209 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
210 template <const int NUM_THREADS = 256>
211 __device__ float block_reduce_sum(float val) {
212     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
213     int warp = threadIdx.x / WARP_SIZE;
214     int lane = threadIdx.x % WARP_SIZE;
215     static __shared__ float shared[NUM_WARPS];
216
217     float value = warp_reduce_sum<WARP_SIZE>(val);
218     if (lane == 0)
219         shared[warp] = value;
220     __syncthreads();
221     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
222     value = warp_reduce_sum<NUM_WARPS>(value);
223     // 关键: broadcast 结果到所有线程, 后续用 result
224     // 做除法等操作时所有线程都能拿到
225     value = __shfl_sync(0xffffffff, value, 0, 32);
226     return value;
227 }
228
229 // Block Reduce Max — FP32 (增强版, 带 broadcast)
230 template <const int NUM_THREADS = 256>
231 __device__ float block_reduce_max(float val) {
232     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
233     int warp = threadIdx.x / WARP_SIZE;
234     int lane = threadIdx.x % WARP_SIZE;
235     static __shared__ float shared[NUM_WARPS];
236
237     float value = warp_reduce_max<WARP_SIZE>(val);
238     if (lane == 0)
239         shared[warp] = value;
240     __syncthreads();
241     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
242     value = warp_reduce_max<NUM_WARPS>(value);
243     value = __shfl_sync(0xffffffff, value, 0, 32);
244     return value;
245 }
246
247 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
248 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
249 // 跨 block 求和的常见模式, 适合 N 较大时使用;
250 // Grid: ((N + 255) / 256, 1, 1)

```

```

251 // Block: (256, 1, 1)
252 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
253 template <const int NUM_THREADS = 256>
254 __global__ void block_reduce_all(float *a, float *y, int N) {
255     int tid = threadIdx.x;
256     int idx = blockIdx.x * NUM_THREADS + tid;
257     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
258     __shared__ float reduce_smem[NUM_WARPS];
259
260     float sum = (idx < N) ? a[idx] : 0.0f;
261     int warp = tid / WARP_SIZE;
262     int lane = tid % WARP_SIZE;
263
264     sum = warp_reduce_sum<WARP_SIZE>(sum);
265     if (lane == 0)
266         reduce_smem[warp] = sum;
267     __syncthreads();
268
269     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
270     if (warp == 0)
271         sum = warp_reduce_sum<NUM_WARPS>(sum);
272     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
273         atomicAdd(y, sum);
274 }
275
276 // ---- Dot Product: y = sum(a[i] * b[i]) ----
277 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
278 // Grid: ((N + 255) / 256, 1, 1)
279 // Block: (256, 1, 1)
280 // source: LeetCUDA/kernels/dot-product/dot_product.cu
281 template <const int NUM_THREADS = 256>
282 __global__ void dot(float *a, float *b, float *y, int N) {
283     int tid = threadIdx.x;
284     int idx = blockIdx.x * NUM_THREADS + tid;
285     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
286     __shared__ float reduce_smem[NUM_WARPS];
287
288     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
289     int warp = tid / WARP_SIZE;
290     int lane = tid % WARP_SIZE;
291
292     prod = warp_reduce_sum<WARP_SIZE>(prod);
293     if (lane == 0)
294         reduce_smem[warp] = prod;
295     __syncthreads();
296
297     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
298     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
299         prod = warp_reduce_sum<NUM_WARPS>(prod);
300     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
301         atomicAdd(y, prod);
302 }
303
304 // Dot Product + float4
305 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
306 // Block: (64, 1, 1), 256/4=64
307 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
308 // source: LeetCUDA/kernels/dot-product/dot_product.cu
309 template <const int NUM_THREADS = 256 / 4>
310 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
311     int tid = threadIdx.x;
312     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
313     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;

```

```

314 __shared__ float reduce_smem[NUM_WARPS];
315
316 float4 reg_a = FLOAT4(a[idx]);
317 float4 reg_b = FLOAT4(b[idx]);
318 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
319                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
320                          : 0.0f;
321 int warp = tid / WARP_SIZE;
322 int lane = tid % WARP_SIZE;
323
324 prod = warp_reduce_sum<WARP_SIZE>(prod);
325 if (lane == 0)
326     reduce_smem[warp] = prod;
327 __syncthreads();
328
329 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
330 if (warp == 0)
331     prod = warp_reduce_sum<NUM_WARPS>(prod);
332 if (tid == 0)
333     atomicAdd(y, prod);
334 }
335
336
337 // =====
338 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
339 // =====
340 // 面试要点:
341 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
342 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
343 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
344
345 // ---- ReLU: y = max(0, x) ----
346 // Grid: ((N + 255) / 256, 1, 1)
347 // Block: (256, 1, 1)
348 // source: LeetCUDA/kernels/relu/relu.cu
349 __global__ void relu(float *x, float *y, int N) {
350     int idx = blockIdx.x * blockDim.x + threadIdx.x;
351     if (idx < N)
352         y[idx] = fmaxf(0.0f, x[idx]);
353 }
354
355 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
356 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
357 // Grid: ((N + 255) / 256, 1, 1)
358 // Block: (64, 1, 1)
359 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
360 // source: LeetCUDA/kernels/relu/relu.cu
361 __global__ void relu_vec4(float *x, float *y, int N) {
362     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
363     if (idx < N) {
364         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
365         float4 reg_y;
366         reg_y.x = fmaxf(0.0f, reg_x.x);
367         reg_y.y = fmaxf(0.0f, reg_x.y);
368         reg_y.z = fmaxf(0.0f, reg_x.z);
369         reg_y.w = fmaxf(0.0f, reg_x.w);
370         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
371     }
372 }
373
374 // ---- Elementwise Add: c = a + b ----
375 // Grid: ((N + 255) / 256, 1, 1)
376 // Block: (256, 1, 1)

```

```

377 // source: LeetCUDA/kernels/elementwise/elementwise.cu
378 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
379     int idx = blockIdx.x * blockDim.x + threadIdx.x;
380     if (idx < N)
381         c[idx] = a[idx] + b[idx];
382 }
383
384 // Elementwise Add + float4 向量化
385 // Grid: ((N + 255) / 256, 1, 1)
386 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
387 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
388 // source: LeetCUDA/kernels/elementwise/elementwise.cu
389 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
390     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
391     if ((idx + 3) < N) {
392         float4 reg_a = FLOAT4(a[idx]);
393         float4 reg_b = FLOAT4(b[idx]);
394         float4 reg_c;
395         reg_c.x = reg_a.x + reg_b.x;
396         reg_c.y = reg_a.y + reg_b.y;
397         reg_c.z = reg_a.z + reg_b.z;
398         reg_c.w = reg_a.w + reg_b.w;
399         FLOAT4(c[idx]) = reg_c;
400     } else if (idx < N) {
401         for (int i = 0; (idx + i) < N; i++) {
402             c[idx + i] = a[idx + i] + b[idx + i];
403         }
404     }
405 }
406
407 // ---- Histogram: y[a[i]]++ ----
408 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
409 // Grid: ((N + 255) / 256, 1, 1)
410 // Block: (256, 1, 1)
411 // source: LeetCUDA/kernels/histogram/histogram.cu
412 __global__ void histogram(int *a, int *y, int N) {
413     int idx = blockIdx.x * blockDim.x + threadIdx.x;
414     if (idx < N)
415         atomicAdd(&(y[a[idx]]), 1);
416 }
417
418 // =====
419 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
420 // =====
421 // 面试要点:
422 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
423 //   online (1-pass, 增量更新)
424 //   - Online Softmax 是 FlashAttention 的数学基础
425 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
426 //   + variance)
427 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
428
429 // =====
430 // Phase 3a: Softmax — 三级递进 (面试核心考点)
431 // =====
432 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
433 // 回答线索: naive(溢出) → safe → online
434
435 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
436 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
437 // 问题: x 值很大时 exp(x) 溢出为 inf
438 template <const int NUM_THREADS = 256>
439 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL

```



```

440 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
441 // source: LeetCode/kernels/softmax/softmax.cu
442 __global__ void softmax_per_token(float *x, float *y, int N) {
443     const int tid = threadIdx.x;
444     const int idx = blockIdx.x * blockDim.x + tid;
445
446     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
447     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
448     if (idx < N)
449         y[idx] = exp_val / exp_sum;
450 }
451
452 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
453 // 面试重点: 为什么 Softmax 需要 Safe?
454 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
455 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
456 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
457 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
458 // Grid: (S, 1, 1)
459 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
460 // source: LeetCode/kernels/softmax/softmax.cu
461 template <const int NUM_THREADS = 256>
462 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
463     const int tid = threadIdx.x;
464     const int idx = blockIdx.x * blockDim.x + tid;
465
466     // Pass 1: block reduce max — 找最大值
467     float val = (idx < N) ? x[idx] : -FLT_MAX;
468     float max_val = block_reduce_max<NUM_THREADS>(val);
469
470     // Pass 2: exp(x - max) → block reduce sum
471     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
472     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
473
474     if (idx < N)
475         y[idx] = exp_val / exp_sum;
476 }
477
478 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
479 // 面试重点: Online Softmax 为什么重要?
480 //   - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
481 //   - Online Softmax 用递推公式合并为 1 遍遍历
482 //   - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
483 //   - 正是 FlashAttention 中 "online rescaling":
484 //       
$$O_{\text{new}} = \text{diag}(\exp(m_{\text{old}} - m_{\text{new}})) * O_{\text{old}} + \exp(m_{\text{cur}} - m_{\text{new}}) * P@V$$

485 // 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
486
487 // ---- Online Softmax 辅助结构 ----
488 // MD struct: 存储 running max (m) 和 running denominator (d = Σexp(x - m))
489 //
490 // 两种使用场景 (公式不同, 但结果等价):
491 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
492 //       m_new = max(m_old, x_i)
493 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
494 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
495 //       m = max(m1, m2)
496 //       d = d1*exp(m1-m) + d2*exp(m2-m)
497 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
498 //
499 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
500 struct __align__(8) MD {
501     float m; // running max
502     float d; // running denominator (sum of exp(x - max))

```

```

503 };
504
505 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
506 //
507 // 不同于单元元素增量更新公式 ( $m_{\text{new}} = \max(m_{\text{old}}, x_i)$ ;  $d_{\text{new}} = d_{\text{old}} \cdot \exp(m_{\text{old}} - m_{\text{new}}) + \exp(x_i - m_{\text{new}})$ ) ,
508 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
509 // (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
510 // (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
511 //
512 // 合并公式 (对称, 不依赖"新/旧"概念) :
513 //  $m = \max(m1, m2)$ 
514 //  $d = d1 \cdot \exp(m1 - m) + d2 \cdot \exp(m2 - m) \leftarrow m1 \geq m2$  时退化为  $d1 + d2 \cdot \exp(m2 - m1)$ 
515 //
516 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
517 template <const int kWarpSize = WARP_SIZE>
518 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
519 #pragma unroll
520     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
521         MD other;
522         other.m = __shfl_xor_sync(0xffffffff, value.m, mask, kWarpSize);
523         other.d = __shfl_xor_sync(0xffffffff, value.d, mask, kWarpSize);
524
525         bool value_bigger = (value.m > other.m);
526         MD bigger_m = value_bigger ? value : other;
527         MD smaller_m = value_bigger ? other : value;
528
529         // d_bigger 无需 rescale (其基准 max 已是全局 max) ,
530         // d_smaller 需 rescale:  $\exp(m_{\text{smaller}} - m_{\text{bigger}})$ 
531         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
532         value.m = bigger_m.m;
533     }
534     return value;
535 }
536
537 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
538 // Grid: (S, 1, 1)
539 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
540 // source: LeetCUDA/kernels/softmax/softmax.cu
541 template <const int NUM_THREADS = 256>
542 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
543     int local_tid = threadIdx.x;
544     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
545     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
546     int warp_id = local_tid / WARP_SIZE;
547     int lane_id = local_tid % WARP_SIZE;
548
549     // 初始化: 每个线程持有一个 (max, denom) 对
550     MD val;
551     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
552     val.d = global_tid < N ? 1.0f : 0.0f;
553
554     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
555     __shared__ MD shared[WARP_NUM];
556     MD res = warp_reduce_md_op<WARP_SIZE>(val);
557
558     if (lane_id == 0)
559         shared[warp_id] = res;
560     __syncthreads();
561
562     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
563     // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
564     if (local_tid < WARP_SIZE) {
565         MD block_res =

```

```

566     local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
567     block_res = warp_reduce_md_op<WARP_NUM>(block_res);
568     if (local_tid == 0) {
569         shared[0] = block_res;
570     }
571 }
572 __syncthreads();
573
574 // 用全局 max 和 denom 做最终 softmax
575 MD final_res = shared[0];
576 float d_total_inverse = __fdividef(1.0f, final_res.d);
577 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
578 if (global_tid < N) {
579     y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
580 }
581 }
582
583 // =====
584 // Phase 3b: RMS Normalization (1-pass reduce)
585 // =====
586 // 面试要点:
587 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
588 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
589 //   - Llama 系列使用 RMS Norm
590 //   - grid(N, K/K), block(K): 一行一个 block
591 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
592 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
593 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
594 template <const int NUM_THREADS = 128>
595 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
596     int tid = threadIdx.x;
597     int bid = blockIdx.x;
598     int idx = bid * blockDim.x + threadIdx.x;
599     const float epsilon = 1e-5f;
600
601     __shared__ float s_variance;
602     float value = (idx < N * K) ? x[idx] : 0.0f;
603     float variance = value * value;
604     variance = block_reduce_sum<NUM_THREADS>(variance);
605     if (tid == 0)
606         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
607     __syncthreads();
608     if (idx < N * K)
609         y[idx] = (value * s_variance) * g;
610 }
611
612 // RMS Norm + float4
613 // Grid: (N, 1, 1)
614 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
615 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
616 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
617 template <const int NUM_THREADS = 128 / 4>
618 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
619     int tid = threadIdx.x;
620     int bid = blockIdx.x;
621     int idx = (bid * blockDim.x + threadIdx.x) * 4;
622     const float epsilon = 1e-5f;
623
624     __shared__ float s_variance;
625     float4 reg_x = FLOAT4(x[idx]);
626     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
627                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
628                                     : 0.0f;

```

```

629 variance = block_reduce_sum<NUM_THREADS>(variance);
630 if (tid == 0)
631     s_variance = rsqrtf(variance / (float)K + epsilon);
632 __syncthreads();
633 float4 reg_y;
634 reg_y.x = reg_x.x * s_variance * g;
635 reg_y.y = reg_x.y * s_variance * g;
636 reg_y.z = reg_x.z * s_variance * g;
637 reg_y.w = reg_x.w * s_variance * g;
638 if (idx < N * K)
639     FLOAT4(y[idx]) = reg_y;
640 }
641
642 // =====
643 // Phase 3c: Layer Normalization (2-pass reduce)
644 // =====
645 // 面试要点:
646 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
647 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
648 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
649 // Grid: (N, 1, 1), 一行一个 block
650 // Block: (128, 1, 1), NUM_THREADS=K=128
651 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
652 template <const int NUM_THREADS = 128>
653 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
654     int tid = threadIdx.x;
655     int bid = blockIdx.x;
656     int idx = bid * blockDim.x + threadIdx.x;
657     const float epsilon = 1e-5f;
658
659     __shared__ float s_mean;
660     __shared__ float s_variance;
661     float value = (idx < N * K) ? x[idx] : 0.0f;
662
663     // Pass 1: compute mean
664     float sum = block_reduce_sum<NUM_THREADS>(value);
665     if (tid == 0)
666         s_mean = sum / (float)K;
667     __syncthreads(); // 必须等待 s_mean 对所有线程可见
668
669     // Pass 2: compute variance = (x - mean)2
670     float variance = (value - s_mean) * (value - s_mean);
671     variance = block_reduce_sum<NUM_THREADS>(variance);
672     if (tid == 0)
673         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
674     __syncthreads(); // 必须等待 s_variance 对所有线程可见
675
676     if (idx < N * K)
677         y[idx] = ((value - s_mean) * s_variance) * g + b;
678 }
679
680 // Layer Norm + float4
681 // Grid: (N, 1, 1)
682 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
683 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
684 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
685 template <const int NUM_THREADS = 128 / 4>
686 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
687                                 int K) {
688     int tid = threadIdx.x;
689     int bid = blockIdx.x;
690     int idx = (bid * blockDim.x + threadIdx.x) * 4;
691     const float epsilon = 1e-5f;

```

```

692
693 __shared__ float s_mean;
694 __shared__ float s_variance;
695 float4 reg_x = FLOAT4(x[idx]);
696 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
697
698 float sum = block_reduce_sum<NUM_THREADS>(value);
699 if (tid == 0)
700     s_mean = sum / (float)K;
701 __syncthreads();
702
703 float4 reg_x_hat;
704 reg_x_hat.x = reg_x.x - s_mean;
705 reg_x_hat.y = reg_x.y - s_mean;
706 reg_x_hat.z = reg_x.z - s_mean;
707 reg_x_hat.w = reg_x.w - s_mean;
708 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
709                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
710 variance = block_reduce_sum<NUM_THREADS>(variance);
711 if (tid == 0)
712     s_variance = rsqrtf(variance / (float)K + epsilon);
713 __syncthreads();
714
715 float4 reg_y;
716 reg_y.x = reg_x_hat.x * s_variance * g + b;
717 reg_y.y = reg_x_hat.y * s_variance * g + b;
718 reg_y.z = reg_x_hat.z * s_variance * g + b;
719 reg_y.w = reg_x_hat.w * s_variance * g + b;
720 if (idx < N * K)
721     FLOAT4(y[idx]) = reg_y;
722 }
723
724 // =====
725 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
726 // =====
727 // 面试要点:
728 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
729 //     [x1']   [cos(θ)  -sin(θ)] [x1]
730 //     [x2'] = [sin(θ)   cos(θ)] [x2]
731 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
732 //   - token_pos = idx / N: token 在序列中的位置
733 //   - token_idx = idx % N: token 内的维度对索引
734 //   - 输入 [seq_len, hidden_size], 输出同形状
735
736 // Grid: ((seq_len * N + 255) / 256, 1, 1)
737 // Block: (256, 1, 1)
738 // source: LeetCUDA/kernels/rope/rope.cu
739 __global__ void rope(float *x, float *out, int seq_len, int N) {
740     int idx = blockIdx.x * blockDim.x + threadIdx.x;
741     float x1 = x[idx * 2];
742     float x2 = x[idx * 2 + 1];
743     int token_pos = idx / N; // 序列位置
744     int token_idx = idx % N; // 维度对索引
745
746     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
747     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
748     float sin_v = sinf(token_pos * exp_v);
749     float cos_v = cosf(token_pos * exp_v);
750
751     // 2D 旋转
752     float out1 = x1 * cos_v - x2 * sin_v;
753     float out2 = x1 * sin_v + x2 * cos_v;
754     out[idx * 2] = out1;

```

```

755     out[idx * 2 + 1] = out2;
756 }
757
758 // =====
759 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
760 // =====
761 // 面试要点 (Bank Conflict 专题) :
762 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
763 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
764 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
765 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
766 //
767 // 转置的四步演进:
768 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
769 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
770 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
771 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
772 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
773
774 // 每个线程处理 1 个元素, block(16,16)
775 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
776 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
777 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
778 // Block: (16, 16, 1)
779 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
780 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
781     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
782     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
783     if (r < row && c < col) {
784         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
785         y[c * row + r] = x[r * col + c];
786     }
787 }
788
789 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
790 // Block: (16, 16, 1)
791 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
792 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
793 __global__ void mat_transpose_padded(
794     float *x, float *y, const int row, const int col) {
795     const int tx = threadIdx.x, ty = threadIdx.y;
796
797     constexpr int TILE = 16;
798     constexpr int PAD = 1;
799     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
800     __shared__ float tile[TILE * 4][TILE + PAD];
801
802     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
803     const int x_c = blockIdx.x * TILE + tx;
804     const int x_r = blockIdx.y * TILE + ty;
805
806     if (x_r * 4 < row && x_c < col) {
807         // Step 1: 4 rows × 1 col → smem (row-major);
808         #pragma unroll
809         for (int i = 0; i < 4; ++i) {
810             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
811         }
812         __syncthreads();
813
814         // Step 2: Transposed read from smem
815         float4 reg_trans;
816         reg_trans.x = tile[tx * 4 + 0][ty];
817         reg_trans.y = tile[tx * 4 + 1][ty];

```

```

818     reg_trans.z = tile[tx * 4 + 2][ty];
819     reg_trans.w = tile[tx * 4 + 3][ty];
820
821     // y 空间坐标: y_r = x 的列, y_c = x 的行
822     const int y_r = blockIdx.x * TILE + ty;          // = x col
823     const int y_c = (blockIdx.y * TILE + tx) * 4;    // = x row
824
825     // Coalesced write: y[y_r][y_c .. y_c+3]
826     FLOAT4(y[y_r * row + y_c]) = reg_trans;
827 }
828 }
829
830 // =====
831 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
832 // =====
833 // 面试要点:
834 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
835 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
836 //   - 不同 K 值对应不同分块策略:
837 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
838 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
839 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
840 //
841 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
842
843 // ---- SGEMV K32: 基础 warp-per-row ----
844 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
845 // grid(M/4), 每个 warp 负责一行
846 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
847 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
848 // Block: (32, 4, 1), 每行 1 warp
849 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
850 // source: LeetCUDA/kernels/sgemv/sgemv.cu
851 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
852     int tx = threadIdx.x; // 0~31
853     int ty = threadIdx.y; // 0~3
854     int bx = blockIdx.x;
855     int lane = tx % WARP_SIZE; // 0~31
856     int m = bx * blockDim.y + ty; // 全局行号
857     if (m < M) {
858         float sum = 0.0f;
859         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
860         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
861         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
862 #pragma unroll
863         for (int w = 0; w < NUM_ITERS; ++w) {
864             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
865             int k = w * WARP_SIZE + lane;
866             sum += a[m * K + k] * x[k];
867         }
868         sum = warp_reduce_sum<WARP_SIZE>(sum);
869         // 每个 warp 处理一行, lane 0 写回结果
870         if (lane == 0)
871             y[m] = sum;
872     }
873 }
874
875 // ---- SGEMV K128: float4 向量化 ----
876 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
877 // Grid: ((M + 3) / 4, 1, 1)
878 // Block: (32, 4, 1)
879 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
880 // source: LeetCUDA/kernels/sgemv/sgemv.cu

```



```

881 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
882     int tx = threadIdx.x; // 0~31
883     int ty = threadIdx.y; // 0~3
884     int bx = blockIdx.x;
885     int lane = tx % WARP_SIZE; // 0~31
886     int m = blockDim.y * bx + ty;
887
888     if (m < M) {
889         float sum = 0.0f;
890         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
891         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
892 #pragma unroll
893         for (int w = 0; w < NUM_ITERS; ++w) {
894             int k = (w * WARP_SIZE + lane) * 4;
895             float4 reg_x = FLOAT4(x[k]);
896             float4 reg_a = FLOAT4(a[m * K + k]);
897             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
898                 reg_a.w * reg_x.w);
899         }
900         sum = warp_reduce_sum<WARP_SIZE>(sum);
901         if (lane == 0)
902             y[m] = sum;
903     }
904 }
905
906 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
907 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
908 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
909 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
910 // Block: (32, 4, 1)
911 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
912 // source: LeetCUDA/kernels/sgemv/sgemv.cu
913 template <const int ROW_PER_WARP = 2>
914 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
915     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
916     int tx = threadIdx.x;
917     int ty = threadIdx.y;
918     int bx = blockIdx.x;
919     int lane = tx % WARP_SIZE;
920     int k = lane % K_WARP_SIZE; // 0~15
921     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
922     if (m < M) {
923         float sum = A[m * K + k] * x[k];
924         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
925         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
926         // 注意: 判断条件是 k == 0, 不是 lane == 0!
927         if (k == 0)
928             y[m] = sum;
929     }
930 }
931
932 // =====
933 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
934 // =====
935 // 面试要点 (GEMM 优化五层金字塔):
936 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
937 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
938 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
939 //   load/store 指令数 Level 4 — Tensor Core (MMA
940 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
941 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
942 //
943 // 计算密度递进:

```



```

944 // Level 1: AI  $\approx$  B_K / (2*sizeof)  $\approx$  32/8 = 4  $\rightarrow$  仍是 memory-bound
945 // Level 2: AI  $\approx$  TM*TN*B_K / (2*sizeof)  $\approx$  8*8*8/8 = 64  $\rightarrow$  compute-bound
946 // Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp  $\rightarrow$  大幅提升吞吐
947
948 // =====
949 // Phase 7a: SGEMM + HGEMM (非 Tensor Core 路径)
950 // =====
951
952 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
953 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
954 // C = A x B, C[M, N] = A[M, K] x B[K, N]
955 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
956 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
957 // Block: (32, 32, 1), 1024 线程
958 // source: LeetCUDA/kernels/sgemm/sgemm.cu
959 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
960     constexpr int BM = 32; // vec 版: 32x4 = 128
961     constexpr int BN = 32; // vec 版: 32x4 = 128
962     constexpr int BK = 32;
963     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
964
965     int bx = blockIdx.x;
966     int by = blockIdx.y;
967     int tx = threadIdx.x;
968     int tid = threadIdx.y * blockDim.x + tx;
969
970     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
971     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4), row 0~127, [128x32]
972     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4, t 0~7, col
973         0~31, 每个线程加载 4 个元素, 8x4 = 32
974     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
975     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每
976         个线程加载 4 个元素, 32x4 = 128
977     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续)
978         [128x128]
979     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)
980
981     float sum = 0.f; // 遍历完整的K, slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小, 才能保证32x32线程处理[32
982         x4, 32x4]=[128x128]大小
983     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
984         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
985         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
986         // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
987         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
988         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
989         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
990         // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
991         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
992         __syncthreads(); // 确保整个 smem tile 加载完毕
993     }
994
995     #pragma unroll
996     for (int k = 0; k < BK; ++k) {
997         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
998         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
999         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1000     }
1001     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1002
1003     int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1004     int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1005     int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1006     c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1007 }

```

```

1003 // ---- Level 1+: SGEMM Vec4 — Block Tile 128×128 + K Tile 32 + Thread Tile 4×4 ----
1004 // 在 Level 1 基础上引入两层优化:
1005 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1006 // 2) Thread Tile 4×4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1007 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1008 // C = A × B, C[M, N] = A[M, K] × B[K, N], A/B 均 row-major
1009 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4×4=16 个 C 元素
1010 // 1024 × 16 = 16384 = 128 × 128 ✓
1011 //
1012 //
1013 // 线程到 4×4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1014 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1015 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1016 //
1017 // 加载映射 (每线程 4 个元素, float4):
1018 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1019 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1020 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1021 //
1022 // △ Bank Conflict 提示 (面试加分点):
1023 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1024 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1025 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1026 //
1027 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1028 // Block: (32, 32, 1), 1024 线程
1029 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1030 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1031 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1032     constexpr int BM = 128;
1033     constexpr int BN = 128;
1034     constexpr int BK = 32;
1035     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1036     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1037
1038     int bx = blockIdx.x;
1039     int by = blockIdx.y;
1040     int tx = threadIdx.x;
1041     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1042
1043     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1044     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1045     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1046     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1047     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1048     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1049
1050     int load_gmem_a_m = by * BM + load_smem_a_m;
1051     int load_gmem_b_n = bx * BN + load_smem_b_n;
1052
1053     // 4×4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1054     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1055     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1056
1057     float sum[4][4] = {0.f};
1058     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1059         int load_gmem_a_k = bk * BK + load_smem_a_k;
1060         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1061         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1062         int load_gmem_b_k = bk * BK + load_smem_b_k;
1063         int load_gmem_b_addr = load_gmem_b_n * N + load_gmem_b_k;
1064         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1065         __syncthreads();

```

```

1066
1067 #pragma unroll
1068     for (int k = 0; k < BK; ++k) {
1069         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1070         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1071                             s_a[comp_smem_a_m_base + 1][k],
1072                             s_a[comp_smem_a_m_base + 2][k],
1073                             s_a[comp_smem_a_m_base + 3][k]};
1074         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1075                             s_b[k][comp_smem_b_n_base + 1],
1076                             s_b[k][comp_smem_b_n_base + 2],
1077                             s_b[k][comp_smem_b_n_base + 3]};
1078     #pragma unroll
1079         for (int i = 0; i < 4; ++i) {
1080             #pragma unroll
1081             for (int j = 0; j < 4; ++j) {
1082                 sum[i][j] += a_vals[i] * b_vals[j];
1083             }
1084         }
1085     }
1086     __syncthreads();
1087 }
1088
1089 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1090 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1091 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1092 #pragma unroll
1093 for (int i = 0; i < 4; ++i) {
1094     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1095     float4 reg_c;
1096     reg_c.x = sum[i][0];
1097     reg_c.y = sum[i][1];
1098     reg_c.z = sum[i][2];
1099     reg_c.w = sum[i][3];
1100     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1101 }
1102 }
1103
1104 // =====
1105 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1106 // =====
1107 // 面试要点:
1108 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1109 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1110 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1111 //     寄存器)
1112 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1113 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1114 //
1115 // ★ TN 布局详解 (面试高频考点):
1116 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1117 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1118 //   N = Normal (列优先, BLAS 原生格式)
1119 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1120 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1121 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1122 //     BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1123 //     视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1124 //     调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1125 //     T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1126 //     N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1127 //
1128 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B

```

```

1129 //      T = row-major (行优先, 对 BLAS 来说是 transposed)
1130 //      N = col-major (列优先, BLAS native = normal)
1131 //
1132 //      LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1133 //      - A: row-major [M, K], B: row-major [K, N]
1134 //      - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1135 //      - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1136 //
1137 //      TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1138 //      - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1139 //      - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1140 //      - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1141 //      MMA 指令: mma.sync.aligned.m16n8k16.row.col
1142 //      - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1143 //
1144 //      WGMMMA (Warp Group MMA, Hopper+):
1145 //      - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1146 //      - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1147 //      - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1148 //      threads) 做计算
1149 //      - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1150
1151 // =====
1152 // Phase 7b-1: MMA PTX 宏定义
1153 // =====
1154
1155 // ---- gmem → smem: cp.async ----
1156 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1157 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1158 #define CP_ASYNC_WAIT_GROUP(n) \
1159     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1160
1161 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1162 #define CP_ASYNC_CG(dst, src, bytes) \
1163     asm volatile( \
1164         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1165         "l"(src), "n"(bytes))
1166
1167 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1168 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1169 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1170 // aligned: 要求 128-bit 对齐, trans: 转置加载
1171 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1172     asm volatile( \
1173         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1174         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1175         : "r"(addr))
1176
1177 #define LDMATRIX_X2(R0, R1, addr) \
1178     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1179         : "=r"(R0), "=r"(R1) \
1180         : "r"(addr))
1181
1182 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1183 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1184 #define LDMATRIX_X2_T(R0, R1, addr) \
1185     asm volatile( \
1186         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1187         : "=r"(R0), "=r"(R1) \
1188         : "r"(addr))
1189
1190 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1191 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)

```

```

1192 // row.col: A 是 row-major, B 是 col-major
1193 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1194 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1195 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1196 #define MMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1197     asm volatile( \
1198         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1199         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1200         : "=r"(RD0), "=r"(RD1) \
1201         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1202         "r"(RC1)) \
1203
1204 // ---- MMA 辅助函数 ----
1205 // div_ceil: 整数除法向上取整
1206 #define HOST_DEVICE_INLINE __device__ __host__ inline
1207 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1208     return (a % b != 0) ? (a / b + 1) : (a / b);
1209 }
1210
1211 // =====
1212 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1213 // =====
1214 // 面试重点 — Tile Hierarchy:
1215 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1216 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1217 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1218 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1219 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1220 //
1221 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1222 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1223 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1224 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1225 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1226 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1227 //
1228 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1229 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1230 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1231 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1232 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1233 //   - 消除 ~40 行尾端循环重复 (ldmatrix+MMA 只写一次)
1234 //
1235 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1236 // Block: (256, 1, 1), 8 warps
1237 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1238 // =====
1239 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1240           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1241           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1242           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1243 __global__ void __launch_bounds__(256)
1244     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1245     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1246     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1247     const int by = blockIdx.y;
1248     constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1249     constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1250     constexpr int BK = MMA_K; // 16
1251
1252     // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1253     // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1254     // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)

```

```

1255 // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1256 // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1257 extern __shared__ half smem[];
1258 half *s_a = smem;
1259 half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1260 constexpr int s_a_stage_offset = BM * BK; // 128*16
1261 constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1262 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1263 const int warp_id = tid / WARP_SIZE; // 0~7
1264 const int lane_id = tid % WARP_SIZE; // 0~31
1265 const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1266 const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1267
1268 // 线程到 global memory 的映射 (用于加载 A 和 B)
1269 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1270 int load_smem_a_m = tid / 2; // 0~127
1271 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1272 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1273 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1274 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1275 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1276 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1277     return;
1278
1279 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1280 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1281 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是VAL Tile的作用。
1282 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1283
1284 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1285 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1286 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1287
1288 // 预加载前 (K_STAGE-1) 个 stage
1289 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1290 #pragma unroll
1291 for (int k = 0; k < (K_STAGE - 1); ++k) {
1292     int load_gmem_a_k = k * BK + load_smem_a_k;
1293     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1294     int load_gmem_b_k = k * BK + load_smem_b_k;
1295     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1296     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1297
1298     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1299         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1300     );
1301     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1302
1303     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1304         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1305     );
1306     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1307
1308     CP_ASYNC.COMMIT_GROUP();
1309 }
1310
1311 const int NUM_K_TILES = div_ceil(K, BK);
1312 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1313 __syncthreads();
1314
1315 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1316 #pragma unroll
1317 for (int k = 0; k < NUM_K_TILES; ++k) {

```



```

1318     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1319     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1320
1321     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1322     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1323     if (k + K_STAGE - 1 < NUM_K_TILES) {
1324         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1325         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1326         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1327         // B^T: row-major [n][k], 内维连续的是 K
1328         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1329
1330         uint32_t load_smem_a_ptr =
1331             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1332                 load_smem_a_m * BK + load_smem_a_k) *
1333                 sizeof(half));
1334         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1335
1336         uint32_t load_smem_b_ptr =
1337             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1338                 load_smem_b_n * BK + load_smem_b_k) *
1339                 sizeof(half));
1340         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1341         CP_ASYNC.COMMIT_GROUP();
1342     }
1343
1344     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1345     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1346     // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1347     uint32_t RA[VAL_TILE_M][4];
1348     uint32_t RB[VAL_TILE_N][2];
1349
1350     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1351     #pragma unroll
1352     for (int i = 0; i < VAL_TILE_M; ++i) {
1353         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1354         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1355         // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1356         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1357         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1358         uint32_t lane_smem_a_ptr =
1359             (smem_a_base_ptr +
1360                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1361                 sizeof(half));
1362         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1363     }
1364
1365     // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1366     // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1367     // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1368     #pragma unroll
1369     for (int j = 0; j < VAL_TILE_N; ++j) {
1370         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1371         int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1372         // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1373         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1374         int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1375         uint32_t lane_smem_b_ptr =
1376             (smem_b_base_ptr +
1377                 (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1378                 sizeof(half));
1379         LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1380     }

```

```

1381 // MMA compute: 发射 VAL_TILE_M × VAL_TILE_N 条 MMA 指令
1382 #pragma unroll
1383 for (int i = 0; i < VAL_TILE_M; ++i) {
1384 #pragma unroll
1385     for (int j = 0; j < VAL_TILE_N; ++j) {
1386         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1387                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1388                 RB[j][0], RB[j][1], // B fragment
1389                 RC[i][j][0], RC[i][j][1]);
1390     }
1391 }
1392
1393 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1394 if (k + K_STAGE - 1 < NUM_K_TILES) {
1395     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1396 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1397     CP_ASYNC_WAIT_GROUP(0);
1398 }
1399 __syncthreads();
1400 }
1401
1402 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1403 {
1404     for (int i = 0; i < VAL_TILE_M; ++i) {
1405         uint32_t RC0[VAL_TILE_N][4]; // 32 bits × 4 = 128 bits = 8 half
1406         uint32_t RC1[VAL_TILE_N][4]; // 32 bits × 4 = 128 bits = 8 half
1407         // =====
1408         // MMA m16n8k16 C fragment — a single 16×8 matrix (registers per warp):
1409         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1410         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1411         //
1412         //      cols: c0-1    c2-3    c4-5    c6-7
1413         //      r0:  T0{c0,c1}  T1      T2      T3      |
1414         //      r1:  T4      T5      T6      T7      |
1415         //      r2:  T8      →  T9 →  T10 →  T11      | RC[0]
1416         //      ...
1417         //      r7:  T28      T29      T30      T31      |
1418         //      r8:  T0{c2,c3}  T1      T2      T3      |
1419         //      r9:  T4      T5      T6      T7      |
1420         //      r10: T8      →  T9 →  T10 →  T11      | RC[1]
1421         //      ...
1422         //      r15: T28      T29      T30      T31      |
1423         //
1424         // Within a 4-lane group (e.g. T0-T3 for row 0):
1425         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1426         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1427         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1428         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store。
1429         // =====
1430     }
1431     #pragma unroll
1432     for (int j = 0; j < VAL_TILE_N; ++j) {
1433         RC0[j][0] = RC[i][j][0];
1434         RC1[j][0] = RC[i][j][1];
1435         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1436         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1437         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1438         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1439         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1440         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1441     }
1442     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1443     // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):

```



```

1444 //
1445 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1446 // |-----|-----|-----|-----|
1447 // | 0~3   | 0       | row 0    | row 8    |
1448 // | 4~7   | 1       | row 1    | row 9    |
1449 // | 8~11  | 2       | row 2    | row 10   |
1450 // | 12~15 | 3       | row 3    | row 11   |
1451 // | 16~19 | 4       | row 4    | row 12   |
1452 // | 20~23 | 5       | row 5    | row 13   |
1453 // | 24~27 | 6       | row 6    | row 14   |
1454 // | 28~31 | 7       | row 7    | row 15   |
1455 // |-----|-----|-----|-----|
1456 if (lane_id % 4 == 0) {
1457     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1458     int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1459     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1460 #pragma unroll
1461     for (int j = 0; j < VAL_TILE_N; ++j) {
1462         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1463         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1464         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1465         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1466         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1467         // 128-bit store: 一次写入 8 个 half
1468         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1469             *reinterpret_cast<float4*>(&RC0[j][0]);
1470         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1471             *reinterpret_cast<float4*>(&RC1[j][0]);
1472     }
1473 }
1474 }
1475 }
1476 }
1477
1478 // =====
1479 // Phase 7c: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1480 // =====
1481 // 面试要点 (WGMMMA vs MMA 对比) :
1482 //   - MMA: warp 级 (32 threads) , 同步执行
1483 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
1484 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
1485 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1486 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
1487 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1488
1489 // ---- WGMMMA 辅助函数 ----
1490 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1491 #define WGMMMA_COMMIT_GROUP() \
1492     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1493 #define WGMMMA_WAIT_GROUP(n) \
1494     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
1495
1496 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1497 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
1498 //   encoded = (x & 0x3FFFF) >> 4
1499 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1500 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1501 // 可省略以扩大可表示范围。
1502 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
1503 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1504
1505 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1506 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,

```

```

1507 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1508 //
1509 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
1510 // -----
1511 // | 位域          | 范围      | 大小 | 含义
1512 // |-----|-----|-----|-----
1513 // | start_address  | [0, 14)   | 14   | smem 基址编码
1514 // | (unused)       | [14, 16)  | 2    | -
1515 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
1516 // | (unused)       | [30, 32)  | 2    | -
1517 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
1518 // | (unused)       | [46, 49)  | 3    | -
1519 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
1520 // | (unused)       | [52, 62)  | 10   | -
1521 // | layout_type    | [62, 64)  | 2    | swizzle 模式
1522 // -----
1523 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
1524 //
1525 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
1526 // - 128B swizzle atom 在 128-bit 单位下为 8×8
1527 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
1528 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
1529 // - 这是 stride_byte_offset=1024 的来源
1530 //
1531 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
1532 //   此处填 16 仅为占位, 编码后为 1。
1533 //
1534 // - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。
1535 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1536 //
1537 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1538 __device__ inline uint64_t make_smem_desc(half *ptr) {
1539 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1540 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
1541 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1542
1543 // 从全零开始: 所有位域初始化为 0。
1544 uint64_t desc = 0x0000000000000000;
1545
1546 // — bits [0, 14): start_address —
1547 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
1548 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1549 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
1550 desc |= SMEM_DESC_ENCODE(addr);
1551
1552 // — bits [16, 30): leading_byte_offset —
1553 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
1554 // << 16 将编码值放到 bits [16, 30)。
1555 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
1556 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1557 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1558 //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
1559 //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
1560 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1561
1562 // — bits [32, 46): stride_byte_offset —
1563 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
1564 // << 32 将编码值放到 bits [32, 46)。
1565 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
1566 // 1024 的来源 (K-Major + 128B swizzle + half) :
1567 //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
1568 //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
1569 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。

```

```

1570 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1571
1572 // — bits [62, 64): layout_type (swizzle mode) —
1573 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1574 // 1 = 128B swizzle mode。
1575 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1576 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
1577 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1578 desc |= 1llu << 62;
1579
1580 return desc;
1581 }
1582
1583 // ---- WGMMMA PTX 指令宏 ----
1584 // wmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1585 // m64n128k16: M=64, N=128, K=16
1586 // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1587 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
1588 // 每线程 64 half = 32 uint32
1589 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1590 // ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
1591 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
1592                                     TransB) \
1593 { \
1594     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1595     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1596     asm volatile( \
1597         "{\n" \
1598         "wmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1599         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
1600         "%8, %9, %10, %11, %12, %13, %14, %15, " \
1601         "%16, %17, %18, %19, %20, %21, %22, %23, " \
1602         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
1603         "%32, " \
1604         "%33, " \
1605         "%34, %35, %36, %37, %38;\n" \
1606         "}\n" \
1607         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1608         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1609         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1610         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1611         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1612         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1613         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1614         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1615         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
1616         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1617         "n"(int32_t(TransB))); \
1618 }
1619
1620 // ---- TMA Shared Memory Layout ----
1621 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1622 //
1623 // 每个 stage 包含两块 smem:
1624 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1625 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1626 // 注意 B 的 smem 布局是 row-major [BK, BN], 物理上按 K-major 排列 (imm-trans-b=0),
1627 // 与 A 的 K-major 配合供 WGMMMA 读取。128B swizzle 将 [BK,BN] row-major 重新映射为
1628 // K-major 布局, 使得 K 维元素在 128B atom 内连续, 区别于 MMA TN 布局下的 B^T[N,K]。
1629 template <int BM, int BN, int BK, int QSIZE> struct WmmaSMem {
1630     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1631     // B: K-major 布局, 供 WGMMMA imm-trans-b=0 直接读取, [BK, BN]。
1632     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN]

```

```

1633 };
1634
1635 // ---- WGMMA Kernel: Warp Specialization + TMA ----
1636 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化) :
1637 //
1638 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1639 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMA 支持将工作拆分为两个
1640 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1641 //
1642 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1643 //   将 A/B tile 从 HBM 搬到 shared memory。
1644 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
1645 //
1646 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1647 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪
1648 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可被覆盖
1649 //
1650 // 本节重点理解:
1651 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1652 //      即可搬运整个 2D tile, 无需所有线程参与。
1653 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1654 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1655 //      Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
1656 //
1657 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比) :
1658 //   WGMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1659 //   K Tile:          BK=64, 每个 K tile 包含 BK/WGMMA_K=4 个 WGMMA atom
1660 //   M Tile:          BM=128, 每个 M tile 包含 BM/WGMMA_M=2 个 WGMMA atom
1661 //   N Tile:          BN=128, 单个 WGMMA_N=128 即可覆盖, 无需在 N 方向分块
1662 //   Block Tile:      C[128,128] = A[128,64] × B[64,128]
1663 //   Threads:         256 = 2 warpgroups × 128 threads/warpgroup
1664 //   Producer(WG0):   128 threads (仅 thread 0 做 TMA 提交)
1665 //   Consumer(WG1):   128 threads = 4 warps (全部参与 WGMMA 和写回)
1666 //
1667 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
1668 // 的延迟被计算完全掩盖。
1669 //
1670 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1671 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
1672 // Block: (256, 1, 1), 2 warpgroups
1673 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1674 template <const int WGMMA_M = 64, const int WGMMA_N = 128,
1675           const int WGMMA_K = 16, const int BM = 128, const int BN = 128,
1676           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1677           const bool BLOCK_SWIZZLE = false>
1678 __global__ void __launch_bounds__(NUM_THREADS)
1679   hgemm_wgmma_stages_tn(
1680     int M, int N, int K, half *C,
1681     const CUtensorMap *__restrict__ tensorMapA,
1682     const CUtensorMap *__restrict__ tensorMapB) {
1683
1684   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1685   // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1686   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1687   // 不展开宿主侧 create_tensor_map 细节。
1688
1689   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1690   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1691   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1692   const int by = blockIdx.y;
1693   // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1694   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1695   constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG

```

```

1696 // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1697 // 当只有一个 consumer 时, 它负责全部 BM=128 行
1698 constexpr int B_WG_M = BM / num_consumers; // 128
1699
1700 // 边界检查: 确保当前 block 不超出 M/N 范围
1701 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1702     return;
1703
1704 // ---- Shared Memory 分配 ----
1705 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1706 // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1707 extern __shared__ __align__(128) uint8_t smem_AB[];
1708 WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1709     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_AB);
1710 half *s_a = s.A;
1711 half *s_b = s.B;
1712
1713 // ---- cuda::barrier 初始化 ----
1714 // 每个 stage 有两个 barrier:
1715 // full[qidx]: Producer thread 0 发 full 信号, 128 个 Consumer 线程等 full
1716 // empty[qidx]: Consumer 发 empty 信号, Producer thread 0 等 empty
1717 // 每轮参与 arrive 的总人数 = 128 (consumer) + 1 (producer) = 129
1718 #pragma nv_diag_suppress static_var_with_dynamic_init
1719 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1720 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1721 #pragma nv_diag_default static_var_with_dynamic_init
1722
1723 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
1724 const int num_blocks_k = K / BK;
1725 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1726 const int tid = threadIdx.x % 128; // 0~127 within warpgroup
1727
1728 // 初始化 barriers (仅 thread 0 执行)
1729 if (threadIdx.x == 0) {
1730     for (int i = 0; i < K_STAGE; ++i) {
1731         // init 的第二个参数是 barrier 的 arrive count:
1732         // num_consumers * 128 + 1 = 1 * 128 + 1 = 129
1733         // 即每轮需要 128 个 consumer 线程 + 1 个 producer 线程都 arrive 后,
1734         // barrier 才翻转 phase 并唤醒等待线程。
1735         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1736         init(&empty[i], num_consumers * 128 + 1); // same
1737     }
1738     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1739     // 对 async proxy (TMA/WGMMMA) 可见。
1740     cuda::device::experimental::fence_proxy_async_shared_cta();
1741 }
1742 __syncthreads();
1743
1744 // =====
1745 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1746 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
1747 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
1748 // =====
1749 if (wg_idx == 0) {
1750     if (tid == 0) {
1751         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1752         int qidx = 0;
1753         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1754              ++block_k_iter, ++qidx) {
1755             if (qidx == K_STAGE)
1756                 qidx = 0;
1757
1758             // Step P1: 等待 Consumer 释放此 stage (empty 信号)

```

```

1759 // empty[qidx].arrive() 表示 Producer 自己"已准备好等待",
1760 // 然后 wait() 阻塞直到 barrier phase 翻转 (即所有 Consumer 都已 arrive)。
1761 empty[qidx].wait(empty[qidx].arrive());
1762
1763 // Step P2: 提交 TMA 2D 拷贝指令
1764 // cp_async_bulk_tensor_2d_global_to_shared:
1765 // 参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1766 // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1767 // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1768 // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1769 //
1770 // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
1771 // 无需线程逐元素搬运, 零寄存器开销。
1772 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1773 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1774     &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1775     full[qidx]);
1776
1777 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1778 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1779     &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1780     full[qidx]);
1781
1782 // Step P3: 通知 Consumer 此 stage 已准备好 (full 信号)
1783 // barrier_arrive_tx 除了 arrive 之外还额外告知硬件:
1784 // 本次 TMA 事务预期传输的字节数。
1785 // Consumer 的 full[qidx].wait() 会等待这 (BK*BN+BK*BM)*sizeof(half)
1786 // 字节全部写入 smem 后才返回。
1787 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
1788     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1789 }
1790 }
1791 }
1792 // =====
1793 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1794 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1795 // 所有 128 个线程 (4 warps) 全部参与。
1796 // =====
1797 else {
1798     // Step C0: Consumer 初始时"准备就绪"
1799     // 对所有 stage 的 empty barrier 执行 arrive, 表示初始时所有 stage
1800     // 都是"空的" (可被 Producer 写入)。
1801     // 如果没有这一步, Producer 的 empty[qidx].wait() 在第一轮会永远阻塞。
1802     for (int i = 0; i < K_STAGE; ++i) {
1803         [[maybe_unused]] auto token = empty[i].arrive();
1804     }
1805
1806     // 累加器寄存器声明
1807     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
1808     // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
1809     // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
1810     // - d[*][g][*]: N 方向第 g 组 16 列
1811     // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
1812     // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
1813     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
1814     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1815
1816     int qidx = 0;
1817     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
1818     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1819         ++block_k_iter, ++qidx) {
1820         if (qidx == K_STAGE)
1821             qidx = 0;

```



```

1822 // Step C1: 等待 Producer 的 full 信号
1823 // 表示 stage qidx 的 A/B tile 数据已通过 TMA 搬运到 smem。
1824 full[qidx].wait(full[qidx].arrive());
1825
1826 // Step C2: 发射 WGMMMA 指令序列
1827 //
1828 // WGMMMA 指令序列的固定模式:
1829 //   WGMMMA_FENCE -> 发射一串 WGMMMA -> COMMIT_GROUP -> WAIT_GROUP
1830 //
1831 // WGMMMA_FENCE: 确保 smem 写可见、accum 寄存器准备好。
1832 //   是一条内存 fence 指令, 建立 async proxy 与 generic proxy 之间的
1833 //   可见性顺序。必须在发射 WGMMMA 之前执行。
1834 //
1835 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
1836 //   执行  $D[64*128] += A[64*16] * B[16*128]$ 
1837 //   其中 A 的 smem 指针由 descA 描述 (K-major), B 由 descB 描述 (K-major)
1838 //   imm-trans-a=0/imm-trans-b=0 表示 A/B 都是 K-major (非转置)。
1839 //   ScaleD=1: 累加模式 (不清零), 用于 K 维累加。
1840 //   ScaleA=ScaleB=1: 不翻转符号。
1841 WGMMMA_FENCE();
1842
1843 // M 维迭代:  $BM/WGMMMA\_M = 128/64 = 2$  个 WGMMMA atom
1844 // 每个 atom 处理 64 行 M 方向数据。
1845 #pragma unroll
1846 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1847     // wmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
1848     // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
1849     half *wmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1850
1851     // K 维迭代:  $BK/WGMMMA\_K = 64/16 = 4$  次 WGMMMA (累加)
1852     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
1853 #pragma unroll
1854 for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1855     // 第 k_it 次 WGMMMA:
1856     //   A: wmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
1857     //   B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
1858     //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
1859     //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
1860     //   ScaleD=1: 累加 (K 维迭代需要累积结果)
1861     WGMMMA_M64N128K16_F16F16F16(d[m_it], wmma_sA + k_it * WGMMMA_K,
1862                                   s_b + qidx * BK * BN + k_it * WGMMMA_K,
1863                                   1, // ScaleD=1: accumulate (不清零)
1864                                   1, 1, 0, 0);
1865 }
1866 }
1867
1868 // Step C3: 提交并等待 WGMMMA 完成
1869 // COMMIT_GROUP: 将此前发射的所有 WGMMMA 指令归为一组提交。
1870 // WAIT_GROUP(0): 等待之前 commit 的所有 WGMMMA 完成 (0 表示等待所有组)。
1871 // 注意 WGMMMA 是异步执行 (async proxy), 这些指令用于同步 completion。
1872 WGMMMA_COMMIT_GROUP();
1873 WGMMMA_WAIT_GROUP(0);
1874
1875 // Step C4: 释放此 stage (发 empty 信号给 Producer)
1876 // 通知 Producer stage qidx 的 smem 已使用完毕, 可以安全覆盖。
1877 [[maybe_unused]] auto token = empty[qidx].arrive();
1878 }
1879
1880 // =====
1881 // Epilogue: 将寄存器中的累加结果写回 global memory C
1882 // =====
1883
1884

```



```

1885 // WGMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
1886 //
1887 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
1888 // 这些 half 分布在 d[2][8][4] 数组中:
1889 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
1890 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
1891 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
1892 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
1893 //
1894 // 其中:
1895 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
1896 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
1897 //
1898 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
1899 // +-----+-----+
1900 // | reg[0] | reg[2] | <- rows [row, row+8)
1901 // |(col,+2)|(col+8)|
1902 // +-----+-----+
1903 // | reg[1] | reg[3] | <- rows [row+8, row+16)
1904 // |(col,+2)|(col+8)|
1905 // +-----+-----+
1906 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
1907 //
1908 // 三维遍历:
1909 // m_it=0: rows 0~63, m_it=1: rows 64~127
1910 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
1911 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
1912 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
1913
1914 const int lane = tid % 32;
1915 const int warp = tid / 32;
1916 // row: 当前线程在当前 WGMMA atom 中负责的起始行
1917 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]
1918 const int row = warp * 16 + lane / 4;
1919 // block_C: 当前 C tile 在 global memory 中的起始地址
1920 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
1921 half *block_C = C + by * BM * N + bx * BN;
1922
1923 #pragma unroll
1924 for (int m_it = 0; m_it < B_WG_M / WGMMA_M; ++m_it) {
1925     int yo = m_it * WGMMA_M; // M 方向行偏移 (0 或 64)
1926 #pragma unroll
1927     for (int g = 0; g < WGMMA_N / 16; ++g) {
1928         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
1929         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
1930         // 左上象限: (row, col)
1931         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1932             d[m_it][g][0];
1933         // 左下象限: (row+8, col)
1934         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1935             d[m_it][g][1];
1936         // 右上象限: (row, col+8)
1937         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1938             d[m_it][g][2];
1939         // 右下象限: (row+8, col+8)
1940         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
1941             d[m_it][g][3];
1942     }
1943 }
1944 }
1945 }
1946
1947 #if (defined(NOTES_V2_HAS_WGMMA) \

```

```

1948 || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
1949 || defined(__CUDA_ARCH_FEAT_SM90_ALL))
1950 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
1951 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled) :
1952 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
1953 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
1954 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
1955 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
1956 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
1957 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
1958 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
1959 //
1960 // 参考: CUDA Programming Guide $TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
1961
1962 template <int BlockMajorSize, int BlockMinorSize>
1963 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
1964                                               half *gmem_ptr,
1965                                               int blocks_height,
1966                                               int blocks_width) {
1967     void *gmem_address = (void *)gmem_ptr;
1968     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
1969                                     (uint64_t)BlockMajorSize * blocks_height,
1970                                     1, 1, 1};
1971     uint64_t gmem_prob_stride[5] = {
1972         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
1973     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
1974                                   uint32_t(BlockMajorSize), 1, 1, 1};
1975     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
1976     CUresult result = cuTensorMapEncodeTiled(
1977         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
1978         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
1979         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
1980         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
1981     if (result != CUDA_SUCCESS)
1982         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
1983 }
1984
1985 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
1986     half *src, int blocks_height, int blocks_width) {
1987     CUtensorMap *tma_map_d;
1988     cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
1989     CUtensorMap tma_map_host;
1990     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
1991     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
1992               cudaMemcpyHostToDevice);
1993     return tma_map_d;
1994 }
1995 #endif /* NOTES_V2_HAS_WGMMMA */
1996
1997 // =====
1998 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
1999 // =====
2000 // 面试要点 (FlashAttention 算法) :
2001 //   1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2002 //     但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2003 //   2. FA 三板斧:
2004 //     a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
2005 //     b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2006 //     c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2007 //   3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2008 //     - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2009 //     - 优点: 减少 warp 间通信和 shuffle
2010 //   4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :

```

```

2011 //      for each K,V tile:
2012 //          S_cur = Q @ K^T                                // 未缩放, 存入 R_S
2013 //          m_new = max(m_old, row_max(S_cur * scale))
2014 //          P_cur = exp(S_cur * scale - m_new)              // ← 写回 R_S 寄存器!
2015 //          l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2016 //          O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2017 //          O_final = O_new / l_final
2018 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2019 //     - R_S 经过 softmax 后, 存储的是 P = exp(S - m), 数据仍然是 half 精度
2020 //     - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2021 //       后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
2022 //     - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
2023 //       都天然同构”的通用结论
2024 //
2025 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
2026 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
2027 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
2028 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
2029 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
2030
2031 // ---- 寄存器填充辅助函数 ----
2032 template <typename T, int M, const int N, const int K = 2>
2033 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
2034 #pragma unroll
2035     for (int i = 0; i < M; ++i)
2036 #pragma unroll
2037         for (int j = 0; j < N; ++j)
2038 #pragma unroll
2039             for (int k = 0; k < K; ++k)
2040                 R[i][j][k] = val;
2041 }
2042
2043 template <typename T, int M, const int N = 2>
2044 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2045 #pragma unroll
2046     for (int i = 0; i < M; ++i)
2047 #pragma unroll
2048         for (int j = 0; j < N; ++j)
2049             R[i][j] = val;
2050 }
2051
2052 // =====
2053 // FlashAttention-2 Split-Q Kernel (完整实现)
2054 // =====
2055 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2056 //
2057 // Tile 设计 (以 kHeadDim=64 为例) :
2058 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
2059 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
2060 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2061 //
2062 // 执行流程:
2063 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2064 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2065 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2066 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
2067 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2068 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2069 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2070 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2071 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2072 //   8) 最终 rescale: O_final = (1/l_final) * O_final
2073 //   9) Epilogue: warp shuffle + 128-bit collective store

```

```

2074
2075 template <
2076     const int kHeadDim,          // head dim: 32, 64, 128
2077     const int kMmaAtomM,        // 16 (MMA instruction M dimension)
2078     const int kMmaAtomN,        // 8 (MMA instruction N dimension)
2079     const int kMmaAtomK,        // 16 (MMA instruction K dimension)
2080     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
2081     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
2082     const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
2083     const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
2084     const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
2085     const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
2086     const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
2087     const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
2088     const int kStage,            // pipeline stages for K: 1 or 2
2089     const int kPad>             // padding for bank conflict avoidance
2090 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
2091     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2092                                     int QKV_seqlen, int QKV_head) {
2093     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
2094     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
2095     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2096     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2097     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2098     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2099     const float scale = 1.0f / sqrtf((float)kHeadDim);
2100
2101     // Block indexing
2102     const int QKV_batch_id = blockIdx.y / QKV_head;
2103     const int QKV_head_id = blockIdx.y % QKV_head;
2104     const int Q_tile_id = blockIdx.x;
2105     const int tid = threadIdx.x;
2106     const int warp_id = tid / WARP_SIZE;
2107     const int lane_id = tid % WARP_SIZE;
2108     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2109     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2110
2111     // Global memory base offsets for this (batch, head)
2112     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2113     const int Q_gmem_offset =
2114         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2115         kHeadDim;
2116     const int K_gmem_offset = Q_gmem_offset;
2117     const int V_gmem_offset = Q_gmem_offset;
2118     const int O_gmem_offset = Q_gmem_offset;
2119
2120     // Thread-to-smem mapping for cooperative load
2121     int load_smem_Q_Br = tid / (kNumThreads / Br);
2122     int load_smem_Q_d =
2123         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2124     int load_smem_K_Bc = tid / (kNumThreads / Bc);
2125     int load_smem_K_d =
2126         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2127     int load_smem_V_Bc = tid / (kNumThreads / Bc);
2128     int load_smem_V_d =
2129         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2130
2131     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2132     if (load_gmem_Q_Br >= QKV_seqlen)
2133         return;
2134
2135     // ---- Shared memory layout ----
2136     extern __shared__ half smem[];

```

```

2137 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2138 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2139 half *Q_tile_smem = smem;
2140 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2141 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2142 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2143 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2144
2145 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2146 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2147 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2148
2149 // ---- Online Softmax persistent state ----
2150 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2151 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2152 float lane_block_row_max_old[kValTileSeqLenQ][2];
2153 float lane_block_row_sum_old[kValTileSeqLenQ][2];
2154 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2155 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2156
2157 // ---- Register allocation ----
2158 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
2159 uint32_t R_K[kValTileSeqLenK][2]; // K regs
2160 uint32_t R_V[kValTileHeadDimV][2]; // V regs
2161 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2162 // 对单个 m16n8k16 tile 而言:
2163 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
2164 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
2165 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2166 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
2167 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
2168 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
2169             [2]; // O accumulator (final output)
2170
2171 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2172 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
2173
2174 // =====
2175 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2176 // =====
2177 {
2178     int load_gmem_Q_addr =
2179         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2180     uint32_t load_smem_Q_ptr =
2181         smem_Q_base_ptr +
2182         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2183 #pragma unroll
2184     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2185         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2186     }
2187     CP_ASYNC_COMMIT_GROUP();
2188 }
2189
2190 // =====
2191 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2192 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
2193 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
2194 // =====
2195 if constexpr (kStage > 1) {
2196 #pragma unroll
2197     for (int stage = 0; stage < (kStage - 1); ++stage) {
2198         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2199         int load_gmem_K_addr =

```

```

2200     K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2201     uint32_t load_smem_K_ptr =
2202         smem_K_base_ptr +
2203         (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2204         load_smem_K_d) *
2205         sizeof(half);
2206 #pragma unroll
2207     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2208         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2209     }
2210     CP_ASYNC_COMMIT_GROUP();
2211 }
2212 CP_ASYNC_WAIT_GROUP(kStage - 2);
2213 __syncthreads();
2214 }
2215
2216 // =====
2217 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2218 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2219 // =====
2220 #pragma unroll 1
2221 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2222     int smem_sel = tile_K_seqlen % kStage;
2223     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2224
2225     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2226     if constexpr (kStage > 1) {
2227         // 只有 kStage>1 才能真正做 K 的 pipeline:
2228         // smem_sel 负责“当前正在计算”的 K tile, smem_sel_next 负责“下一轮预取”的 K tile。
2229         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
2230         // Load current V tile (no pipeline for V — one stage is enough)
2231         {
2232             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2233             int load_gmem_V_addr =
2234                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2235             uint32_t load_smem_V_ptr =
2236                 smem_V_base_ptr +
2237                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2238 #pragma unroll
2239             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2240                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2241             }
2242             CP_ASYNC_COMMIT_GROUP();
2243         }
2244
2245         // Prefetch next K tile (pipelined)
2246         if ((tile_K_seqlen + 1) < Tc) {
2247             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2248             int load_gmem_K_addr =
2249                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2250             uint32_t load_smem_K_ptr =
2251                 smem_K_base_ptr +
2252                 (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2253                 load_smem_K_d) *
2254                 sizeof(half);
2255 #pragma unroll
2256             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2257                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2258             }
2259             CP_ASYNC_COMMIT_GROUP();
2260         }
2261     }
2262 }

```



```

2263 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2264 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2265 #pragma unroll
2266 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2267     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2268     #pragma unroll
2269     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2270         int warp_smem_Q_Br =
2271             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
2272         int lane_smem_Q_Br =
2273             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2274         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2275         uint32_t lane_smem_Q_ptr =
2276             smem_Q_base_ptr +
2277             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2278         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2279             lane_smem_Q_ptr);
2280     }
2281
2282     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2283     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2284     #pragma unroll
2285     for (int j = 0; j < kValTileSeqLenK; ++j) {
2286         int warp_smem_K_Bc =
2287             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
2288         int lane_smem_K_Bc =
2289             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2290         int lane_smem_K_d =
2291             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2292         uint32_t lane_smem_K_ptr =
2293             smem_K_base_ptr +
2294             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2295             lane_smem_K_d) *
2296             sizeof(half);
2297         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2298     }
2299
2300     // MMA: S[tile] += Q[tile] @ K^T[tile]
2301     #pragma unroll
2302     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2303         #pragma unroll
2304         for (int j = 0; j < kValTileSeqLenK; ++j) {
2305             HMMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2306                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2307                 R_S[i][j][1]);
2308         }
2309     }
2310 } // end loop over d
2311 __syncthreads();
2312
2313 // =====
2314 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
2315 // =====
2316 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2317 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2318 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2319 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2320 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2321 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2322 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2323 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
2324 // kValTileSeqLenK tiles.
2325

```



```

2326 float lane_row_max_new[kValTileSeqLenQ][2];
2327 float lane_row_sum_new[kValTileSeqLenQ][2];
2328 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2329 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2330
2331 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
2332 #pragma unroll
2333 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2334 #pragma unroll
2335     for (int j = 0; j < kValTileSeqLenK; ++j) {
2336         // Extract half values from R_S registers (C matrix fragment layout)
2337         float2 t_reg_S_0 =
2338             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2339         float2 t_reg_S_1 =
2340             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2341         // S = (Q@K^T) * scale
2342         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2343         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2344         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2345         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2346     }
2347     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2348     // {0,4,8,...,28} hold valid data)
2349     lane_row_max_new[i][0] =
2350         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2351     lane_row_max_new[i][1] =
2352         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2353 }
2354
2355 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
2356 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2357 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2358 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2359 #pragma unroll
2360 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2361     // m_new = max(m_old, m_cur)
2362     float block_row_max_new_0 =
2363         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2364     float block_row_max_new_1 =
2365         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2366
2367 #pragma unroll
2368     for (int j = 0; j < kValTileSeqLenK; ++j) {
2369         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2370         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2371
2372         // P = exp(S * scale - m_new), 用 fma 保证精度
2373         t_reg_S_0.x =
2374             __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2375         t_reg_S_0.y =
2376             __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2377         t_reg_S_1.x =
2378             __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2379         t_reg_S_1.y =
2380             __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2381
2382         // Accumulate row sums
2383         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2384         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2385
2386         // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2387         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2388         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);

```

```

2389     }
2390
2391     // Warp-level reduce sum (warp_size = 4, same as max)
2392     lane_row_sum_new[i][0] =
2393         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2394     lane_row_sum_new[i][1] =
2395         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2396 }
2397 __syncthreads();
2398
2399 // =====
2400 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
2401 // =====
2402 // Wait for V to be ready before computing P@V
2403 if constexpr (kStage > 1) {
2404     if ((tile_K_seqlen + 1) < Tc) {
2405         CP_ASYNC_WAIT_GROUP(1);
2406     } else {
2407         CP_ASYNC_WAIT_GROUP(0);
2408     }
2409 } else {
2410     CP_ASYNC_WAIT_GROUP(0);
2411 }
2412 __syncthreads();
2413
2414 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
2415
2416 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2417 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2418 #pragma unroll
2419 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2420     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2421     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2422     // transposed ldmatrix
2423 #pragma unroll
2424 for (int j = 0; j < kValTileHeadDimV; ++j) {
2425     int warp_smem_V_d =
2426         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2427     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2428     int lane_smem_V_d = warp_smem_V_d;
2429     uint32_t lane_smem_V_ptr =
2430         smem_V_base_ptr +
2431         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2432     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2433 }
2434
2435 // P matrix layout in R_S[i][j][2]:
2436 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2437 // | 64x64 | warp_KV 0 |
2438 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2439 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2440 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2441 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2442 // tile_V_Bc selects which 16-column slice of P to use:
2443 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2444 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2445 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2446 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2447 // 对应的 MMA A fragment 布局可以这样记:
2448 // - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
2449 // - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
2450 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2451 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA

```

```

2452 // fragment 都无条件成立的通用结论。layout转换逻辑:
2453 // C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
2454 int w = tile_V_Bc * 2;
2455 #pragma unroll
2456 for (int i = 0; i < kValTileSeqLenP; ++i) {
2457 #pragma unroll
2458 for (int j = 0; j < kValTileHeadDimV; ++j) {
2459     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2460              R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
2461              R_V[j][0], R_V[j][1], // B fragment = V
2462              R_0[i][j][0], R_0[i][j][1]); // C fragment output
2463     }
2464 }
2465 } // end for tile_V_Bc
2466 __syncthreads();
2467
2468 // =====
2469 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
2470 // =====
2471 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
2472 #pragma unroll
2473 for (int i = 0; i < kValTileSeqLenP; ++i) {
2474     float block_row_max_new_0 = lane_row_max_new[i][0];
2475     float block_row_max_new_1 = lane_row_max_new[i][1];
2476     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2477     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2478
2479     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2480     float block_row_max_old_1 = lane_block_row_max_old[i][1];
2481
2482     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2483     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2484
2485     // Handle first iteration: m_old = -inf, need to use m_new directly
2486     block_row_max_old_0 =
2487         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2488     block_row_max_old_1 =
2489         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2490
2491     float rescale_o_factor_0 =
2492         __expf(block_row_max_old_0 - block_row_max_new_0);
2493     float rescale_o_factor_1 =
2494         __expf(block_row_max_old_1 - block_row_max_new_1);
2495
2496     // Rescale 0_old + Add P@V in one fused step
2497 #pragma unroll
2498 for (int j = 0; j < kValTileHeadDimV; ++j) {
2499     // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
2500     // reg[0] -> rows 0~7 的 {c0,c1}
2501     // reg[1] -> rows 8~15 的 {c2,c3}
2502     float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2503     float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2504     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2505     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2506
2507     // 0_new = exp(m_old - m_new) * 0_old + P@V (fused multiply-add)
2508     t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2509     t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2510     t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2511     t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2512
2513     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2514     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);

```

```

2515     }
2516
2517     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
2518     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2519     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2520     lane_block_row_sum_old[i][0] = __fmaf_rn(
2521         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2522     lane_block_row_sum_old[i][1] = __fmaf_rn(
2523         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2524
2525     // Update m
2526     lane_block_row_max_old[i][0] = block_row_max_new_0;
2527     lane_block_row_max_old[i][1] = block_row_max_new_1;
2528 }
2529
2530 // Wait for next K tile to be ready in smem before next iteration
2531 if constexpr (kStage > 1) {
2532     if ((tile_K_seqlen + 1) < Tc) {
2533         CP_ASYNC_WAIT_GROUP(0);
2534     }
2535     __syncthreads();
2536 }
2537 } // end outer loop over K seqlen
2538 __syncthreads();
2539
2540 // =====
2541 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
2542 // =====
2543 #pragma unroll
2544 for (int i = 0; i < kValTileSeqLenP; ++i) {
2545     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2546     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2547 #pragma unroll
2548 for (int j = 0; j < kValTileHeadDimV; ++j) {
2549     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2550     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2551     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2552     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2553     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2554     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2555     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2556     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2557 }
2558 }
2559
2560 // =====
2561 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2562 // =====
2563 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2564 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2565 #pragma unroll
2566 for (int i = 0; i < kValTileSeqLenP; ++i) {
2567 #pragma unroll
2568 for (int j = 0; j < kValTileHeadDimV; ++j) {
2569     uint32_t R_Z[2][4];
2570     R_Z[0][0] = R_D[i][j][0];
2571     R_Z[1][0] = R_D[i][j][1];
2572     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2573     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2574     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2575     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2576     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2577     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

```

```

2578 // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2579 if (lane_id % 4 == 0) {
2580     int store_warp_regs_0_Br =
2581         warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
2582     int store_lane_gmem_0_Br =
2583         Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2584     int store_warp_regs_0_d =
2585         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2586     int store_lane_gmem_0_d = store_warp_regs_0_d;
2587     int store_gmem_0_addr_0 = 0_gmem_offset +
2588         (store_lane_gmem_0_Br + 0) * kHeadDim +
2589         store_lane_gmem_0_d;
2590     int store_gmem_0_addr_1 = 0_gmem_offset +
2591         (store_lane_gmem_0_Br + 8) * kHeadDim +
2592         store_lane_gmem_0_d;
2593     // LDST128BITS = reinterpret_cast<float4*>
2594     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2595         *reinterpret_cast<float4*>(&R_Z[0][0]);
2596     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2597         *reinterpret_cast<float4*>(&R_Z[1][0]);
2598 }
2599 }
2600 }
2601 }
2602 }
2603
2604 // =====
2605 // 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
2606 // =====
2607
2608 static inline void check(cudaError_t err, const char *msg) {
2609     if (err != cudaSuccess) {
2610         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2611         exit(EXIT_FAILURE);
2612     }
2613 }
2614
2615 static void test_block_reduce(int N) {
2616     srand(42);
2617     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2618     for (int i = 0; i < N; i++)
2619         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2620
2621     // CPU reference: sum of all elements
2622     double ref = 0.0;
2623     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2624
2625     float *d_a, *d_y;
2626     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2627     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2628
2629     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2630     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2631
2632     dim3 block(128);
2633     dim3 grid((N + 127) / 128);
2634     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
2635     check(cudaGetLastError(), "blockreduce launch");
2636     check(cudaDeviceSynchronize(), "blockreduce sync");
2637
2638     float result;

```

```

2641     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2642
2643     float err = fabsf(result - (float)ref);
2644     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
2645           err < 1e-2f ? "PASS" : "FAIL");
2646
2647     free(h_a);
2648     cudaFree(d_a); cudaFree(d_y);
2649 }
2650
2651
2652 static void test_dot(int N) {
2653
2654     srand(42);
2655     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2656     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2657     for (int i = 0; i < N; i++) {
2658         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2659         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2660     }
2661
2662     // CPU reference
2663     double ref = 0.0;
2664     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2665
2666     float *d_a, *d_b, *d_y;
2667     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2668     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2669     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
2670
2671     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
2672     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2673     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2674
2675     dim3 block(128);
2676     dim3 grid((N + 127) / 128);
2677     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
2678     check(cudaGetLastError(), "dot launch");
2679     check(cudaDeviceSynchronize(), "dot sync");
2680
2681     float result;
2682     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2683
2684     float err = fabsf(result - (float)ref);
2685     printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2686           err < 1e-2f ? "PASS" : "FAIL");
2687
2688     // ---- Dot Vec4 ----
2689     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2690     dim3 block_v4(32);
2691     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
2692     check(cudaGetLastError(), "dot_vec4 launch");
2693     check(cudaDeviceSynchronize(), "dot_vec4 sync");
2694
2695     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2696     float err_v4 = fabsf(result - (float)ref);
2697     printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2698
2699     free(h_a); free(h_b);
2700     cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2701 }
2702
2703

```

```

2704 static void test_relu(int N) {
2705     srand(42);
2706     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2707     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2708     for (int i = 0; i < N; i++)
2709         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2710
2711     float *d_x, *d_y;
2712     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2713     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2714     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2715
2716     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
2717     dim3 block256(256);
2718     dim3 grid256((N + 255) / 256);
2719     relu<<<grid256, block256>>>(d_x, d_y, N);
2720     check(cudaGetLastError(), "relu launch");
2721     check(cudaDeviceSynchronize(), "relu sync");
2722     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2723     float max_err = 0.0f;
2724     for (int i = 0; i < N; i++) {
2725         float expected = fmaxf(0.0f, h_x[i]);
2726         float err = fabsf(h_y[i] - expected);
2727         if (err > max_err) max_err = err;
2728     }
2729     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2730
2731     dim3 block64(64);
2732     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
2733     check(cudaGetLastError(), "relu_vec4 launch");
2734     check(cudaDeviceSynchronize(), "relu_vec4 sync");
2735     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2736     max_err = 0.0f;
2737     for (int i = 0; i < N; i++) {
2738         float expected = fmaxf(0.0f, h_x[i]);
2739         float err = fabsf(h_y[i] - expected);
2740         if (err > max_err) max_err = err;
2741     }
2742     printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2743
2744     free(h_x); free(h_y);
2745     cudaFree(d_x); cudaFree(d_y);
2746 }
2747
2748
2749 static void test_elementwise(int N) {
2750     srand(42);
2751     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2752     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2753     for (int i = 0; i < N; i++) {
2754         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2755         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2756     }
2757
2758     float *d_a, *d_b, *d_c;
2759     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2760     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2761     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2762     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2763     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
2764
2765     dim3 block256(256);
2766     dim3 grid256((N + 255) / 256);

```



```

2767 elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2768 check(cudaGetLastError(), "eadd launch");
2769 check(cudaDeviceSynchronize(), "eadd sync");
2770 float *h_c = (float *)malloc((size_t)N * sizeof(float));
2771 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2772 float max_err = 0.0f;
2773 for (int i = 0; i < N; i++) {
2774     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2775     if (err > max_err) max_err = err;
2776 }
2777 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2778
2779 dim3 block64(64);
2780 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
2781 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
2782 check(cudaGetLastError(), "eadd_vec4 launch");
2783 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
2784 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
2785 max_err = 0.0f;
2786 for (int i = 0; i < N; i++) {
2787     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2788     if (err > max_err) max_err = err;
2789 }
2790 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2791
2792 free(h_a); free(h_b); free(h_c);
2793 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
2794 }
2795
2796
2797 static void test_histogram(int N) {
2798     srand(42);
2799     int BINS = 16;
2800     int *h_hist = (int *)calloc(BINS, sizeof(int));
2801     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
2802     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
2803     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
2804     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
2805
2806     int *d_idx, *d_hist;
2807     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
2808     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
2809     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
2810     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
2811
2812     dim3 block256(256);
2813     dim3 grid256((N + 255) / 256);
2814     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
2815     check(cudaGetLastError(), "histogram launch");
2816     check(cudaDeviceSynchronize(), "histogram sync");
2817     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
2818
2819     float max_err = 0.0f;
2820     for (int i = 0; i < BINS; i++) {
2821         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
2822         if (err > max_err) max_err = err;
2823     }
2824     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2825
2826     free(h_hist); free(h_hist_ref); free(h_idx);
2827     cudaFree(d_idx); cudaFree(d_hist);
2828 }
2829

```

```

2830
2831 static void test_softmax(int N) {
2832     // Use N = blockDim.x so one block processes all elements as one token.
2833     constexpr int NUM_THREADS = 256;
2834     if (N != NUM_THREADS) {
2835         printf("    OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
2836         return;
2837     }
2838
2839     srand(42);
2840     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2841     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
2842     for (int i = 0; i < N; i++)
2843         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
2844
2845     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
2846     float max_val = -FLT_MAX;
2847     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
2848     double sum_exp = 0.0;
2849     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
2850     for (int i = 0; i < N; i++)
2851         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
2852
2853     float *d_x, *d_y;
2854     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
2855     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
2856
2857     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
2858
2859     dim3 block(256);
2860     dim3 grid(1); // one token covering all N elements
2861     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2862     check(cudaGetLastError(), "softmax launch");
2863     check(cudaDeviceSynchronize(), "softmax sync");
2864
2865     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2866     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
2867
2868     float max_err = 0.0f;
2869     for (int i = 0; i < N; i++) {
2870         float err = fabsf(h_y[i] - h_y_ref[i]);
2871         if (err > max_err) max_err = err;
2872     }
2873     printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2874
2875     // ---- Safe Softmax ----
2876     safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2877     check(cudaGetLastError(), "safe_softmax launch");
2878     check(cudaDeviceSynchronize(), "safe_softmax sync");
2879     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
2880     max_err = 0.0f;
2881     for (int i = 0; i < N; i++) {
2882         float err = fabsf(h_y[i] - h_y_ref[i]);
2883         if (err > max_err) max_err = err;
2884     }
2885     printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2886
2887     // ---- Naive Softmax ----
2888     softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2889     check(cudaGetLastError(), "naive_softmax launch");
2890     check(cudaDeviceSynchronize(), "naive_softmax sync");
2891     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
2892     max_err = 0.0f;

```

```

2893     for (int i = 0; i < N; i++) {
2894         float err = fabsf(h_y[i] - h_y_ref[i]);
2895         if (err > max_err) max_err = err;
2896     }
2897     printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2898
2899     free(h_x); free(h_y); free(h_y_ref);
2900     cudaFree(d_x); cudaFree(d_y);
2901 }
2902
2903
2904 static void test_rms_norm(int N, int K) {
2905
2906     srand(42);
2907     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2908     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2909     for (int i = 0; i < N * K; i++)
2910         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2911     float g = 1.5f; // gain
2912
2913     // CPU reference: y = (x / rms(x)) * g
2914     float epsilon = 1e-5f;
2915     for (int n = 0; n < N; n++) {
2916         double sum_sq = 0.0;
2917         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
2918         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
2919         for (int k = 0; k < K; k++)
2920             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
2921     }
2922
2923     float *d_x, *d_y;
2924     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
2925     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
2926     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
2927
2928     dim3 block(128);
2929     dim3 grid(N);
2930     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
2931     check(cudaGetLastError(), "rmsnorm launch");
2932     check(cudaDeviceSynchronize(), "rmsnorm sync");
2933
2934     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2935     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
2936
2937     float max_err = 0.0f;
2938     for (int i = 0; i < N * K; i++) {
2939         float err = fabsf(h_y[i] - h_y_ref[i]);
2940         if (err > max_err) max_err = err;
2941     }
2942     printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2943
2944     // ---- RMS Norm Vec4 ----
2945     dim3 block_rv4(32);
2946     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
2947     check(cudaGetLastError(), "rmsnorm_vec4 launch");
2948     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
2949     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
2950     max_err = 0.0f;
2951     for (int i = 0; i < N * K; i++) {
2952         float err = fabsf(h_y[i] - h_y_ref[i]);
2953         if (err > max_err) max_err = err;
2954     }
2955     printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

```

```

2956     free(h_x); free(h_y); free(h_y_ref);
2957     cudaFree(d_x); cudaFree(d_y);
2958 }
2959
2960
2961
2962 static void test_layer_norm(int N, int K) {
2963
2964     srand(42);
2965     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2966     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2967     for (int i = 0; i < N * K; i++)
2968         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2969     float g = 1.5f, b = 0.3f; // gain and bias
2970
2971     // CPU reference: y = ((x - mean) / std) * g + b
2972     float epsilon = 1e-5f;
2973     for (int n = 0; n < N; n++) {
2974         double sum = 0.0;
2975         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
2976         float mean = (float)sum / (float)K;
2977         double sum_sq = 0.0;
2978         for (int k = 0; k < K; k++) {
2979             float diff = h_x[n * K + k] - mean;
2980             sum_sq += (double)diff * (double)diff;
2981         }
2982         float std = sqrtf((float)sum_sq / (float)K + epsilon);
2983         for (int k = 0; k < K; k++)
2984             h_y_ref[n * K + k] = (h_x[n * K + k] - mean) / std * g + b;
2985     }
2986
2987     float *d_x, *d_y;
2988     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
2989     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
2990     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
2991
2992     dim3 block(128);
2993     dim3 grid(N);
2994     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
2995     check(cudaGetLastError(), "layernorm launch");
2996     check(cudaDeviceSynchronize(), "layernorm sync");
2997
2998     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2999     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
3000
3001     float max_err = 0.0f;
3002     for (int i = 0; i < N * K; i++) {
3003         float err = fabsf(h_y[i] - h_y_ref[i]);
3004         if (err > max_err) max_err = err;
3005     }
3006     printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3007
3008     // ---- Layer Norm Vec4 ----
3009     dim3 block_lv4(32);
3010     layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
3011     check(cudaGetLastError(), "layernorm_vec4 launch");
3012     check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
3013     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
3014     max_err = 0.0f;
3015     for (int i = 0; i < N * K; i++) {
3016         float err = fabsf(h_y[i] - h_y_ref[i]);
3017         if (err > max_err) max_err = err;
3018     }

```

```

3019 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3020
3021 free(h_x); free(h_y); free(h_y_ref);
3022 cudaFree(d_x); cudaFree(d_y);
3023 }
3024
3025
3026 static void test_rope(int seq_len, int N) {
3027
3028     int total_pairs = seq_len * N;
3029     int total_elems = total_pairs * 2;
3030     size_t size = (size_t)total_elems * sizeof(float);
3031
3032     float *h_x = (float *)malloc(size);
3033     float *h_y_ref = (float *)malloc(size);
3034
3035     srand(42);
3036     for (int i = 0; i < total_elems; i++)
3037         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3038
3039     // CPU reference: 2D rotation for each pair
3040     for (int idx = 0; idx < total_pairs; idx++) {
3041         int token_pos = idx / N;
3042         int token_idx = idx % N;
3043         float x1 = h_x[idx * 2];
3044         float x2 = h_x[idx * 2 + 1];
3045         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3046         float angle = (float)token_pos * theta;
3047         float cos_v = cosf(angle);
3048         float sin_v = sinf(angle);
3049         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
3050         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3051     }
3052
3053     float *d_x, *d_y;
3054     check(cudaMalloc(&d_x, size), "rope alloc X");
3055     check(cudaMalloc(&d_y, size), "rope alloc Y");
3056     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3057
3058     dim3 block(256);
3059     dim3 grid((total_pairs + 255) / 256);
3060     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3061     check(cudaGetLastError(), "rope launch");
3062     check(cudaDeviceSynchronize(), "rope sync");
3063
3064     float *h_y = (float *)malloc(size);
3065     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3066
3067     float max_err = 0.0f;
3068     for (int i = 0; i < total_elems; i++) {
3069         float err = fabsf(h_y[i] - h_y_ref[i]);
3070         if (err > max_err) max_err = err;
3071     }
3072     printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3073
3074     free(h_x);
3075     free(h_y);
3076     free(h_y_ref);
3077     cudaFree(d_x);
3078     cudaFree(d_y);
3079 }
3080
3081

```

```

3082 static void test_mat_transpose(int row, int col) {
3083
3084     size_t size_in = (size_t)row * col * sizeof(float);
3085     size_t size_out = (size_t)col * row * sizeof(float);
3086
3087     float *h_x = (float *)malloc(size_in);
3088     float *h_y_ref = (float *)malloc(size_out);
3089
3090     srand(42);
3091     for (int i = 0; i < row * col; i++)
3092         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3093
3094     // CPU reference: y[j][i] = x[i][j] (row-major)
3095     for (int i = 0; i < row; i++)
3096         for (int j = 0; j < col; j++)
3097             h_y_ref[j * row + i] = h_x[i * col + j];
3098
3099     float *d_x, *d_y;
3100     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3101     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3102     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3103
3104     dim3 block(16, 16);
3105     dim3 grid((col + 15) / 16, (row + 15) / 16);
3106     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3107     check(cudaGetLastError(), "mattrans launch");
3108     check(cudaDeviceSynchronize(), "mattrans sync");
3109
3110     float *h_y = (float *)malloc(size_out);
3111     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
3112
3113     float max_err = 0.0f;
3114     for (int i = 0; i < col * row; i++) {
3115         float err = fabsf(h_y[i] - h_y_ref[i]);
3116         if (err > max_err) max_err = err;
3117     }
3118     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3119
3120     free(h_x);
3121     free(h_y);
3122     free(h_y_ref);
3123     cudaFree(d_x);
3124     cudaFree(d_y);
3125 }
3126
3127
3128 static void test_mat_transpose_padded(int row, int col) {
3129
3130     size_t size_in = (size_t)row * col * sizeof(float);
3131     size_t size_out = (size_t)col * row * sizeof(float);
3132
3133     float *h_x = (float *)malloc(size_in);
3134     float *h_y_ref = (float *)malloc(size_out);
3135
3136     srand(42);
3137     for (int i = 0; i < row * col; i++)
3138         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3139
3140     // CPU reference: y[j][i] = x[i][j] (row-major)
3141     for (int i = 0; i < row; i++)
3142         for (int j = 0; j < col; j++)
3143             h_y_ref[j * row + i] = h_x[i * col + j];
3144

```

```

3145 float *d_x, *d_y;
3146 check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3147 check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
3148 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3149
3150 dim3 block(16, 16);
3151 dim3 grid((col + 15) / 16, (row + 63) / 64);
3152 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3153 check(cudaGetLastError(), "mattrans_padded launch");
3154 check(cudaDeviceSynchronize(), "mattrans_padded sync");
3155
3156 float *h_y = (float *)malloc(size_out);
3157 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
3158
3159 float max_err = 0.0f;
3160 for (int i = 0; i < col * row; i++) {
3161     float err = fabsf(h_y[i] - h_y_ref[i]);
3162     if (err > max_err) max_err = err;
3163 }
3164 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3165
3166 free(h_x);
3167 free(h_y);
3168 free(h_y_ref);
3169 cudaFree(d_x);
3170 cudaFree(d_y);
3171 }
3172
3173
3174 static void test_sgemv(int M, int K) {
3175
3176     srand(42);
3177     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3178     float *h_x = (float *)malloc((size_t)K * sizeof(float));
3179     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3180     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3181     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3182
3183     // CPU reference: y[m] = sum_k A[m,k] * x[k]
3184     for (int m = 0; m < M; m++) {
3185         double sum = 0.0;
3186         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3187         h_y_ref[m] = (float)sum;
3188     }
3189
3190     float *d_a, *d_x, *d_y;
3191     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3192     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3193     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3194
3195     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3196     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3197
3198     dim3 block(32, 4);
3199     dim3 grid((M + 3) / 4);
3200     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3201     check(cudaGetLastError(), "sgemv launch");
3202     check(cudaDeviceSynchronize(), "sgemv sync");
3203
3204     float *h_y = (float *)malloc((size_t)M * sizeof(float));
3205     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3206
3207     float max_err = 0.0f;

```



```

3208 for (int m = 0; m < M; m++) {
3209     float err = fabsf(h_y[m] - h_y_ref[m]);
3210     if (err > max_err) max_err = err;
3211 }
3212 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3213
3214 // ---- SGEMV K32 ----
3215 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3216 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3217 check(cudaGetLastError(), "sgemv_k32 launch");
3218 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3219
3220 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
3221
3222 max_err = 0.0f;
3223 for (int m = 0; m < M; m++) {
3224     float err = fabsf(h_y[m] - h_y_ref[m]);
3225     if (err > max_err) max_err = err;
3226 }
3227 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3228
3229 // ---- SGEMV K16 ----
3230 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3231 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3232
3233 int K16 = 16;
3234 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3235 h_x = (float *)malloc((size_t)K16 * sizeof(float));
3236 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3237 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3238 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3239
3240 for (int m = 0; m < M; m++) {
3241     double sum = 0.0;
3242     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3243     h_y_ref[m] = (float)sum;
3244 }
3245
3246 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3247 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3248 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3249
3250 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3251 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3252
3253 dim3 grid_k16((M + 7) / 8);
3254 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3255 check(cudaGetLastError(), "sgemv_k16 launch");
3256 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3257
3258 h_y = (float *)malloc((size_t)M * sizeof(float));
3259 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3260
3261 max_err = 0.0f;
3262 for (int m = 0; m < M; m++) {
3263     float err = fabsf(h_y[m] - h_y_ref[m]);
3264     if (err > max_err) max_err = err;
3265 }
3266 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3267
3268 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3269 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3270 }

```

```

3271
3272
3273 static void test_sgemm(int M, int N, int K) {
3274
3275     size_t size_a = (size_t)M * K * sizeof(float);
3276     size_t size_b = (size_t)K * N * sizeof(float);
3277     size_t size_c = (size_t)M * N * sizeof(float);
3278
3279     float *h_a = (float *)malloc(size_a);
3280     float *h_b = (float *)malloc(size_b);
3281     float *h_c_ref = (float *)malloc(size_c);
3282
3283     srand(42);
3284     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3285     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3286
3287     float *d_a, *d_b, *d_c;
3288     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3289     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3290     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3291
3292     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3293     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3294
3295     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3296     cublasHandle_t handle;
3297     cublasCreate(&handle);
3298     float alpha = 1.0f, beta = 0.0f;
3299     cublasSgemv(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3300                &alpha, d_b, N, d_a, K, &beta, d_c, N);
3301     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
3302
3303     // Kernel
3304     cudaEvent_t start, stop;
3305     cudaEventCreate(&start);
3306     cudaEventCreate(&stop);
3307
3308     dim3 block(32, 32);
3309     dim3 grid((N + 31) / 32, (M + 31) / 32);
3310     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3311     check(cudaGetLastError(), "sgemm launch");
3312     check(cudaDeviceSynchronize(), "sgemm sync");
3313
3314     float *h_c = (float *)malloc(size_c);
3315     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3316
3317     // Verify
3318     float max_err = 0.0f;
3319     for (int i = 0; i < M * N; i++) {
3320         float err = fabsf(h_c[i] - h_c_ref[i]);
3321         if (err > max_err) max_err = err;
3322     }
3323     printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3324
3325     // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3326
3327     dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3328     check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3329     sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3330     check(cudaGetLastError(), "sgemm_vec4 launch");
3331     check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3332
3333     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");

```

```

3334
3335 max_err = 0.0f;
3336 for (int i = 0; i < M * N; i++) {
3337     float err = fabsf(h_c[i] - h_c_ref[i]);
3338     if (err > max_err) max_err = err;
3339 }
3340 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3341
3342 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3343 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3344 cublasDestroy(handle);
3345 cudaEventDestroy(start);
3346 cudaEventDestroy(stop);
3347 }
3348
3349
3350 static void test_hgemmm_mma(int M, int N, int K) {
3351
3352     size_t size_a = (size_t)M * K * sizeof(half);
3353     size_t size_b = (size_t)K * N * sizeof(half);
3354     size_t size_c = (size_t)M * N * sizeof(half);
3355
3356     half *h_a = (half *)malloc(size_a);
3357     half *h_b = (half *)malloc(size_b);
3358     half *h_c_ref = (half *)malloc(size_c);
3359
3360     srand(42);
3361     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3362     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3363
3364     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
3365     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3366     size_t size_b_t = (size_t)N * K * sizeof(half);
3367     half *h_b_t = (half *)malloc(size_b_t);
3368     for (int n = 0; n < N; n++)
3369         for (int k = 0; k < K; k++)
3370             h_b_t[n * K + k] = h_b[k * N + n];
3371
3372     half *d_a, *d_b, *d_b_t, *d_c;
3373     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
3374     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
3375     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
3376     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
3377
3378     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
3379     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
3380     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
3381
3382     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3383     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3384     cublasHandle_t handle;
3385     cublasCreate(&handle);
3386     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3387     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3388                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3389                 &beta_h, d_c, CUDA_R_16F, N,
3390                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3391     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
3392
3393     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3394     cudaEvent_t start, stop;
3395     cudaEventCreate(&start);
3396     cudaEventCreate(&stop);

```

```

3397
3398 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3399 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
3400 dim3 block(256);
3401 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3402 hgemma_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3403 check(cudaGetLastError(), "hgemma launch");
3404 check(cudaDeviceSynchronize(), "hgemma sync");
3405
3406 half *h_c = (half *)malloc(size_c);
3407 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemma D2H");
3408
3409 // Verify
3410 float max_err = 0.0f;
3411 for (int i = 0; i < M * N; i++) {
3412     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3413     if (err > max_err) max_err = err;
3414 }
3415 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3416
3417 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3418 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3419 cublasDestroy(handle);
3420 cudaEventDestroy(start);
3421 cudaEventDestroy(stop);
3422 }
3423
3424
3425 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)
3426 static void test_hgemma_wgmma(int M, int N, int K) {
3427     // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
3428     // TN layout: C[M×N] = A[M×K] × BT[N×K]
3429     // Kernel: hgemma_wgmma_stages_tn with default template params
3430
3431     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
3432
3433     // M, K must be divisible by tile dims
3434     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
3435         printf("| %-35s | %-12s | %-4s |\n",
3436             "HGEMM WGMMA", "SKIP", "SKIP");
3437         return;
3438     }
3439
3440     size_t size_a = (size_t)M * K * sizeof(half);
3441     size_t size_b = (size_t)K * N * sizeof(half);
3442     size_t size_c = (size_t)M * N * sizeof(half);
3443
3444     half *h_a = (half *)malloc(size_a);
3445     half *h_b = (half *)malloc(size_b);
3446     half *h_c_ref = (half *)malloc(size_c);
3447
3448     srand(42);
3449     for (int i = 0; i < M * K; i++)
3450         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3451     for (int i = 0; i < K * N; i++)
3452         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3453
3454     // BT [N×K] row-major for TN layout (same as hgemma_mma kernel)
3455     size_t size_b_t = (size_t)N * K * sizeof(half);
3456     half *h_b_t = (half *)malloc(size_b_t);
3457     for (int n = 0; n < N; n++)
3458         for (int k = 0; k < K; k++)

```

```

3459     h_b_t[n * K + k] = h_b[k * N + n];
3460
3461     half *d_a, *d_b, *d_b_t, *d_c;
3462     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
3463     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
3464     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
3465     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
3466
3467     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
3468     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
3469           "wgmma H2D B (cuBLAS)");
3470     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
3471           "wgmma H2D B_t (kernel)");
3472
3473     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
3474     cublasHandle_t handle;
3475     cublasCreate(&handle);
3476     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3477     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
3478                 CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
3479                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3480     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
3481           "wgmma D2H ref");
3482
3483     // Create TMA tensor maps for A and B^T
3484     // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
3485     // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
3486     CUTensorMap *tma_a =
3487         allocate_and_create_tensor_map(d_a, M / BM, K / BK);
3488     CUTensorMap *tma_b =
3489         allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
3490
3491     // Launch WGMMMA kernel
3492     // BLOCK_SWIZZLE=false → 2D grid, no swizzle
3493     size_t smem_bytes =
3494         K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
3495     cudaFuncSetAttribute(
3496         hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
3497         false>,
3498         cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
3499
3500     dim3 block(NUM_THREADS);
3501     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3502     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
3503         <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
3504     check(cudaGetLastError(), "wgmma launch");
3505     check(cudaDeviceSynchronize(), "wgmma sync");
3506
3507     half *h_c = (half *)malloc(size_c);
3508     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
3509
3510     // Verify
3511     float max_err = 0.0f;
3512     for (int i = 0; i < M * N; i++) {
3513         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3514         if (err > max_err)
3515             max_err = err;
3516     }
3517     printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
3518           max_err < 1.0f ? "PASS" : "FAIL");
3519
3520     free(h_a);
3521     free(h_b);

```

```

3522 free(h_b_t);
3523 free(h_c);
3524 free(h_c_ref);
3525 cudaFree(d_a);
3526 cudaFree(d_b);
3527 cudaFree(d_b_t);
3528 cudaFree(d_c);
3529 cudaFree(tma_a);
3530 cudaFree(tma_b);
3531 cublasDestroy(handle);
3532 }
3533 #endif /* NOTES_V2_HAS_WGMMMA */
3534
3535
3536 static void test_flash_attn(int seqlen, int head_dim) {
3537     // FlashAttention-2 with split-Q, MMA m16n8k16
3538     int B = 1, H = 8;
3539
3540     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
3541
3542     srand(42);
3543     half *h_q = (half *)malloc(sz);
3544     half *h_k = (half *)malloc(sz);
3545     half *h_v = (half *)malloc(sz);
3546     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3547         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3548         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3549         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3550     }
3551
3552     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
3553     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3554     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3555     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3556     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3557     int count = B * H * seqlen * head_dim;
3558     for (int i = 0; i < count; i++) {
3559         ref_q[i] = __half2float(h_q[i]);
3560         ref_k[i] = __half2float(h_k[i]);
3561         ref_v[i] = __half2float(h_v[i]);
3562     }
3563
3564     float scale = 1.0f / sqrtf((float)head_dim);
3565     for (int bi = 0; bi < B * H; bi++) {
3566         for (int qi = 0; qi < seqlen; qi++) {
3567             // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
3568             float smax = -INFINITY;
3569             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3570             for (int kj = 0; kj < seqlen; kj++) {
3571                 float s = 0.0f;
3572                 for (int d = 0; d < head_dim; d++)
3573                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3574                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3575                 S[kj] = s * scale;
3576                 if (S[kj] > smax) smax = S[kj];
3577             }
3578             // softmax
3579             double sum_exp = 0.0;
3580             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3581             float inv_sum = 1.0f / (float)sum_exp;
3582             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
3583             for (int d = 0; d < head_dim; d++) {
3584                 double o_acc = 0.0;

```

```

3585     for (int kj = 0; kj < seqlen; kj++)
3586         o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3587             ref_v[bi * seqlen * head_dim + kj * head_dim + d];
3588     ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3589 }
3590 free(S);
3591 }
3592 }
3593
3594 half *d_q, *d_k, *d_v, *d_o;
3595 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3596 check(cudaMalloc(&d_k, sz), "fa alloc K");
3597 check(cudaMalloc(&d_v, sz), "fa alloc V");
3598 check(cudaMalloc(&d_o, sz), "fa alloc O");
3599 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3600 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3601 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3602
3603 // Template params for kHeadDim=64, kStage=2
3604 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3605 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3606 constexpr int kMmaTileSeqLenQ = 4;
3607 constexpr int kMmaTileSeqLenK = 1;
3608 constexpr int kMmaTileSeqLenP = 4;
3609 constexpr int kMmaTileHeadDimV = 1;
3610 constexpr int kValTileSeqLenQ = 1;
3611 constexpr int kValTileSeqLenK = 8;
3612 constexpr int kValTileSeqLenP = 1;
3613 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
3614
3615 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
3616 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
3617 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3618                     kStageV * Bc * (kHeadDim + kPadV) +
3619                     Bc * (kHeadDim + kPadV)) * sizeof(half);
3620
3621 dim3 block(128);
3622 dim3 grid((seqlen + Br - 1) / Br, B * H);
3623
3624 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3625     kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3626     kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
3627     kStageV, kPadV>
3628     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3629 check(cudaGetLastError(), "fa launch");
3630 check(cudaDeviceSynchronize(), "fa sync");
3631
3632 half *h_o = (half *)malloc(sz);
3633 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3634
3635 float max_err = 0.0f;
3636 for (int i = 0; i < count; i++) {
3637     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3638     if (err > max_err) max_err = err;
3639 }
3640 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3641
3642 free(h_q); free(h_k); free(h_v); free(h_o);
3643 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3644 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3645 }
3646
3647

```



```

3648 int main(int argc, char *argv[]) {
3649 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)
3650     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
3651 #endif
3652     int M = 1024, N = 1024, K = 1024;
3653     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
3654
3655     printf("=== notes-v2.cu verification harness ===\n");
3656     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
3657     printf("|-----|-----|-----|\n");
3658
3659     test_block_reduce(N);
3660     test_dot(N);
3661     test_relu(1024);
3662     test_elementwise(1024);
3663     test_histogram(1024);
3664     test_softmax(256); // softmax kernel requires N == blockDim.x
3665     test_rms_norm(8, 128);
3666     test_layer_norm(8, 128);
3667     test_rope(8, 128);
3668     test_mat_transpose(256, 256);
3669     test_mat_transpose_padded(256, 256);
3670     test_sgemv(256, 128);
3671     test_sgemm(M, N, K);
3672     test_hgemm_mma(M, N, K);
3673 #if defined(NOTES_V2_HAS_WGMMA) \
3674     || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
3675     || defined(__CUDA_ARCH_FEAT_SM90_ALL))
3676     test_hgemm_wgmma(M, N, K);
3677 #endif
3678     test_flash_attn(1024, 64);
3679
3680     printf("=== All tests done ===\n");
3681     return 0;
3682 }
3683
3684 // =====
3685 // Quick build & run reference
3686 // =====
3687 // # sm_89 单独编译 + 运行:
3688 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
3689 //
3690 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
3691 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a -DNOTES_V2_HAS_WGMMA -lcublas -lcuda notes-v2.
    cu -o notes_v2_sm90.bin

```